# 8주차 · Stateflow 미션 — 웨이포인트와 로이터링을 잇는다

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

**★ 이번 학기 녹화 강의 영상은 여기에 올라감 — 수업에서 놓친 부분은 영상으로 확인할 것**

**https://youtube.com/playlist?list=PLK_f1-krzJG8**

- 매주 수업 뒤 그 주차 영상이 이 재생목록에 추가됨
- 아래 "참조 강의" 는 **지난 학기까지의 선수 과목**이고, 이 재생목록은 **이번 학기 본 수업** 녹화임

**★ 참조 강의 — 본 과목이 전제하는 배경**

아래 다섯 과목은 **본 과목 담당 교수가 직접 강의한 것**이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

| # | 과목 | 수준 | 언어 | 영상 | 자료·코드 |
|---|---|---|---|---|---|
| 1 | **제어공학** — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 2 | **제어시스템설계** — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 3 | **캡스톤디자인** — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지 | 학부 | 한국어 | 재생목록 | 드라이브 |
| 4 | **제어공학특론** — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림 | 대학원 | 영어 | 재생목록 | 드라이브 |
| 5 | **센서신호처리 및 융합** — 센서 모델, 잡음, 추정, 다중센서 융합 | 대학원 | 영어 | 재생목록 | 드라이브 |

**MATLAB·Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것.** 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

| 도구 | 시작 지점 |
|---|---|
| MATLAB | MATLAB Onramp · Core MATLAB Skills |
| Simulink | Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부 |

**★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`**

| 저장소 | 주소 | 받는 자리 (WSL) |
|---|---|---|
| 강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트 | **https://github.com/wkyouncnu/Capstone-Design** | `~/Capstone-Design` |
| ROS 2 예제 패키지 `usv_basics` — 2주차부터 쓰는 노드 코드 | **https://github.com/wkyouncnu/usv_basics** | `~/capstone_ws/src/usv_basics` |

| 언제 | 명령 (VS Code 의 WSL 창 터미널) | 하는 일 |
|---|---|---|
| 처음 한 번 | `cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git` | 강의자료 전체를 받음 (약 770 MB · 2026-10-16 실측 1분 40초) |
| 처음 한 번 | `mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git` | 예제 패키지를 받음. 이어서 `cd ~/capstone_ws && colcon build --symlink-install` |
| 매주 수업 전 | `cd ~/Capstone-Design && git pull` · `cd ~/capstone_ws/src/usv_basics && git pull` | **바뀐 파일만** 받음. 다시 clone 하지 않음 |
| 무엇이 바뀌었는지 | `git log --oneline -10` · `git show --stat HEAD` | 교수가 갱신한 내역을 확인 |

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엶
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → 강의자료는-한-번-받고-git-pull-로-갱신한다

- **과목**: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용**: 웨이포인트를 따라가다가 **한 점에서 돌고**, 다시 웨이포인트로 **돌아오게** 만들기

**★ 시작 전 확인**

- **강의자료부터 갱신**: VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull` — 문서와 코드·모델이 같은 판이 됨 (강의자료는-한-번-받고-git-pull-로-갱신한다, 처음 받는 법은 2주차 2-6)
- 5주차 LOS 와 7주차 로이터링이 각각 동작했어야 함
- 배포 폴더 `W08_simulink/`

**ℹ 이번 주차 새로 만드는 것은 하나뿐이다**

유도법칙 두 개(LOS, 벡터필드)와 내부루프·추진기·운동모델은 이미 다 있음
이번 주차에 작성하는 것은 **"언제 무엇을 쓸지 결정하는 것"** — 미션 상태기계

## 이 주차를 마치면 할 수 있어야 하는 것

1. 유한상태기계(FSM)가 무엇이고 왜 필요한지 설명하기
2. Stateflow 로 **상태·전이·전이조건**을 만들기
3. 두 유도법칙을 상태에 따라 갈아 끼우기
4. 로이터 중에 **웨이포인트 인덱스가 넘어가지 않게** 막기
5. **대수 루프**가 왜 생기고 어떻게 끊는지 설명하기
6. 미션 전 구간의 성능을 상태별로 나눠 평가하기
7. 미션 끝에 **접안 상태**를 붙여 유도를 끊고 **위치제어**로 넘기기
8. ㄷ 자 도킹 스테이션에 **순수 옆걸음**으로 붙이고 멈춘 자리를 수치로 검증하기
9. Stateflow 차트가 **0.05 초마다** 입력을 읽고 전이 하나를 실행하는 순서를 설명하기
10. 네 단계(웨이포인트 → 로이터 → 웨이포인트 → 접안)의 전이 조건을 각각 말하고 실측 시각과 맞춰 보기

## 준비물

| 항목 | 내용 |
|---|---|
| 환경 | 1~7주차 그대로 |
| 배포 파일 | `W08_simulink/` (모델 4개 + 스크립트 16개) |
| 파일 갈래 | 실습 A~D 는 `W08_*`, 실습 E(접안) 는 `W08_dock_*` — 서로 영향이 없음 |

---

# 1부 · 이론

## 1-1. 왜 상태기계가 필요한가

- 지금까지 만든 것은 **한 가지 일만 하는 제어기**였음

| 주차 | 하는 일 | 언제 끝나는가 |
|---|---|---|
| 5주차 | 웨이포인트 따라가기 | 마지막 점에 닿으면 |
| 7주차 | 한 점 주위 돌기 | 정한 바퀴 수를 돌면 |

- 실제 임무는 **여러 동작을 순서대로** 함

> 이안 → 항주 → 관측 지점에서 로이터링 → 항주 → 도킹 접근 → 접안

> **★ 이것을 `if` 문으로 짜면 유지보수가 급격히 어려워진다**
> - 동작이 5개면 조건 조합이 수십 가지가 됨
> - "지금 무슨 동작 중인가"가 코드 여기저기에 흩어짐
> - 한 동작을 고치면 다른 동작이 깨짐

### 상태기계라는 답

- **상태(state)** 를 명시적으로 두고, **전이(transition)** 로만 옮겨 다님

| 개념 | 뜻 | 본 절의 예 |
|---|---|---|
| **상태** | 지금 무슨 동작 중인가 | `WpFollow`, `Loiter`, `Finish` |
| **전이** | 상태를 옮기는 화살표 | `WpFollow → Loiter` |
| **전이조건** | 언제 옮기는가 | `[at_loiter > 0.5]` |
| **진입동작** | 상태에 들어갈 때 한 번 | `en: mode = 1;` |

- 한 순간에 **정확히 하나의 상태**만 활성임. 그래서 헷갈릴 여지가 없음

### 상태기계와 Stateflow — 개념과 도구

> **ℹ 둘은 같은 말이 아니다**
> - **상태기계** 는 **개념** — 교과서 · 논문 · 설계 문서에서 쓰는 말
> - **Stateflow** 는 그 개념을 Simulink 안에 **그려 넣는 도구**의 제품명

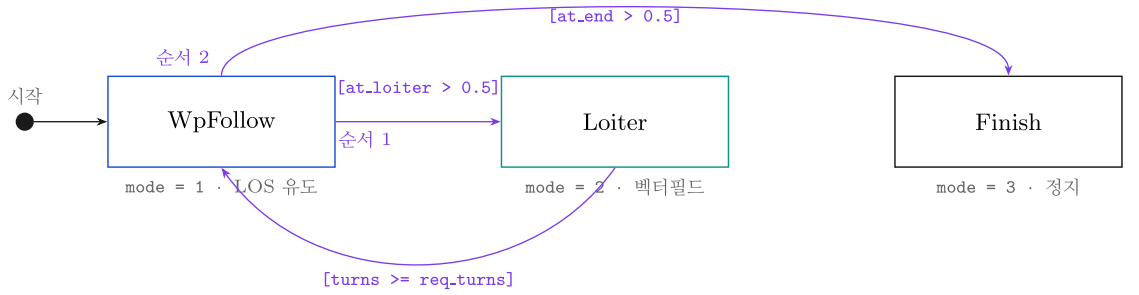
| 용어 | 갈래 | 뜻 |
|---|---|---|
| **상태기계** (state machine) | 개념 | 동작을 상태와 전이로만 나누는 설계 방식 |
| **유한상태기계** (FSM, finite state machine) | 개념 | 상태의 개수가 유한한 상태기계 — 본 과목은 3~4개 |
| **Stateflow** | 도구 | Simulink 에 상태기계를 그리는 MathWorks 제품 |
| **차트** (chart) | 도구 | Stateflow 가 만드는 블록 하나 — 본 과목의 `MissionFSM` |

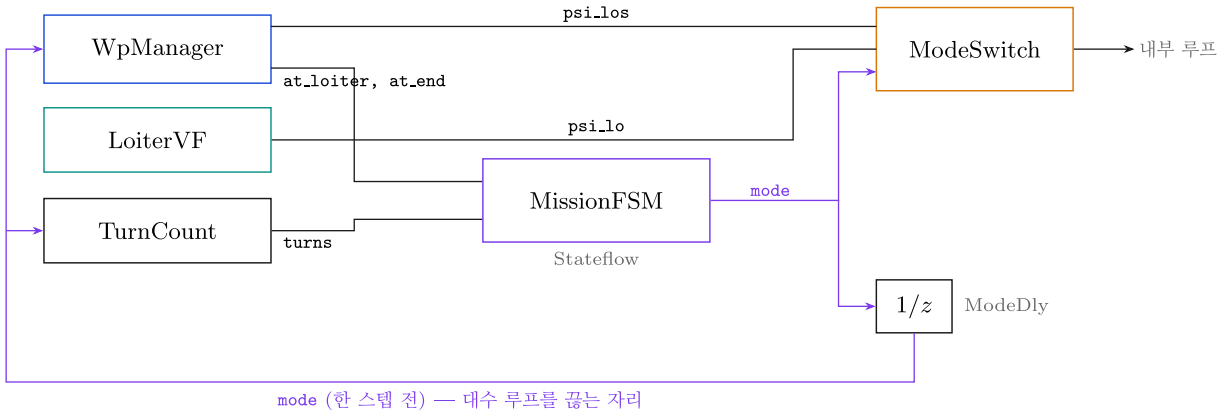
- 본 문서의 표기 — **설계를 말할 때는 상태기계**, **실행과 조작을 말할 때는 Stateflow 차트**
  - 설계: "미션을 네 상태로 나눔"
  - 실행: "차트가 0.05 초마다 전이를 검사함"

- 상태기계는 Stateflow 없이도 성립함 — C 코드로도, MATLAB Function 으로도 짤 수 있음
  - Stateflow 를 쓰는 이유는 **상태와 전이가 눈에 보이고, 실행 중 활성 상태가 표시되기 때문**임

---

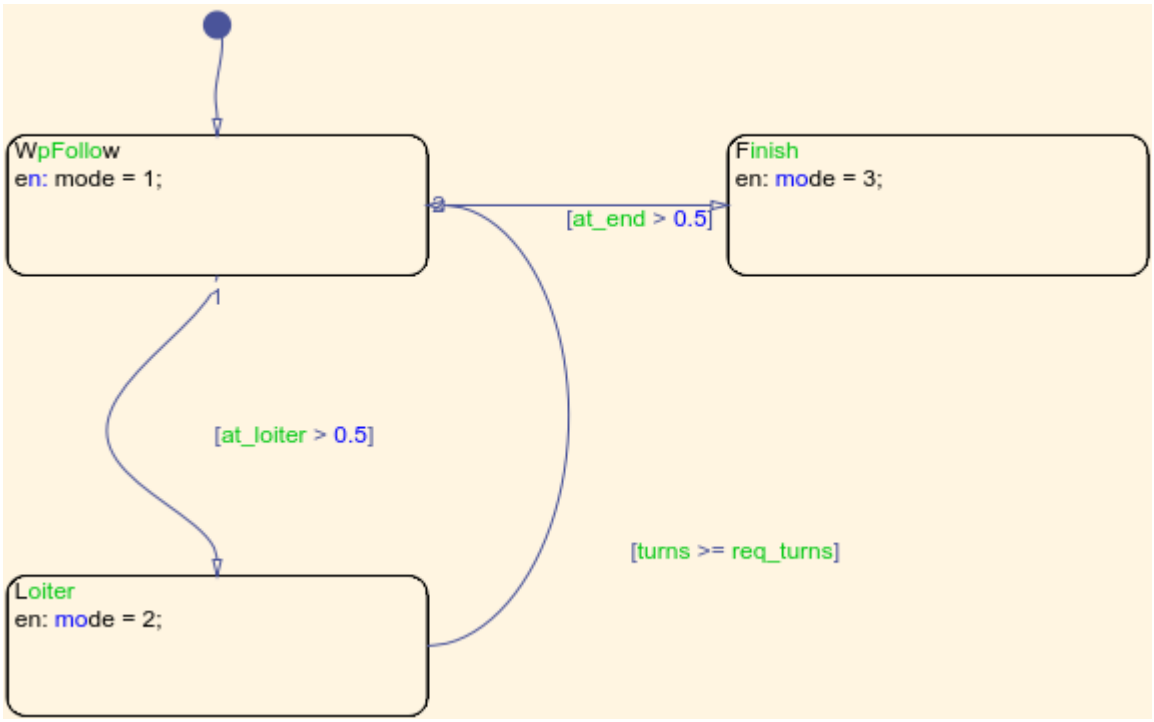
## 1-2. 이번 주차에 작성할 상태기계

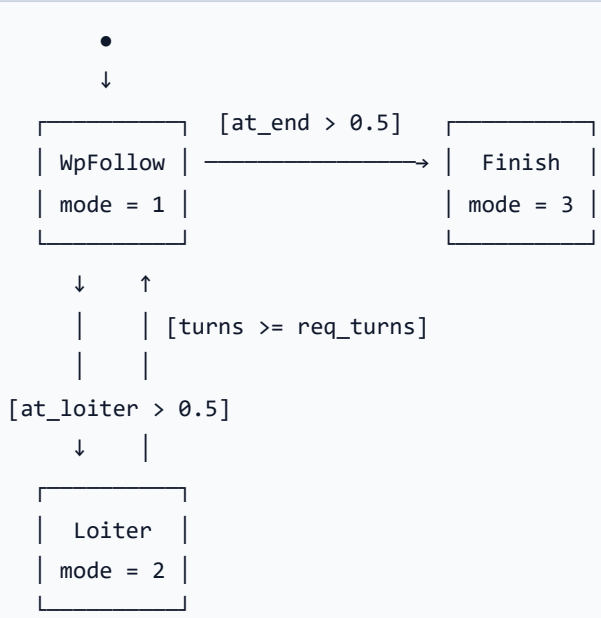



**만들면서 실제로 밟은 문제**

① 로이터 원이 다음 웨이포인트를 지나가면 인덱스가 올라가 그 점을 건너뜀 → 인덱스를 `mode` 로 잠금 (1-4절)
② 로이터 중심을 "현재 웨이포인트" 로 잡으면 인덱스가 움직일 때 원이 통째로 이동 → 중심을 상수로 고정
③ `mode` 되먹임으로 대수 루프 → $1/z$ 로 한 스텝 늦춤 (1-5절)






| 상태 | `mode` | 하는 일 |
|---|---|---|
| `WpFollow` | 1 | LOS 유도로 웨이포인트 따라가기 |
| `Loiter` | 2 | 벡터필드로 원 궤도 돌기 |
| `Finish` | 3 | 정지 |

| 전이 | 조건 | 뜻 |
|---|---|---|
| `WpFollow → Loiter` | `at_loiter > 0.5` | 로이터 지정 웨이포인트에 도착 |
| `Loiter → WpFollow` | `turns >= req_turns` | 정해진 바퀴 수를 다 돎 |
| `WpFollow → Finish` | `at_end > 0.5` | 마지막 웨이포인트에 도착 |

## 도착 신호의 정의

- 두 도착 신호는 `WpManager` 가 계산함. 기호는 아래와 같음
  - $k$ — 지금 달리는 구간의 시작 인덱스 (구간은 WP $k$ → WP $k+1$)
  - $n$ — 웨이포인트 개수 (기본 5), $k_{\text{lo}}$ — `loiter_wp` (기본 3), $R$ — 수락반경 `R_LOS` (5 m)
  - $d_{k+1}$ — 배 $(x, y)$ (코드 `x_n`, `y_n`) 에서 구간 끝 WP $(x_{k+1}, y_{k+1})$ 까지의 거리

$$d_{k+1} = \sqrt{(x_{k+1} - x)^2 + (y_{k+1} - y)^2}$$

$$\texttt{at_loiter} = \begin{cases} 1 & d_{k+1} \le R \text{ 이고 } k+1 = k_{\text{lo}} \\ 0 & \text{그 밖} \end{cases} \qquad \texttt{at_end} = \begin{cases} 1 & d_{k+1} \le R \text{ 이고 } k = n-1 \\ 0 & \text{그 밖} \end{cases}$$

> **i 전이 조건을 `> 0.5` 로 쓰는 이유**
>
> - 도착 신호 `at_loiter`, `at_end` 는 MATLAB Function 이 내는 **0 또는 1 의 double** 임
> - 실수를 `== 1` 로 비교하면 계산 오차에 걸릴 수 있음
> - 그래서 0 과 1 의 한가운데인 0.5 로 가름
> - 부록 A1 M-4 절(SB13)에서 같은 관용구를 작은 모델로 먼저 연습함

## 차트의 데이터 — 입력 4개, 출력 1개

| 이름 | 범위 | 포트 | 어디서 오는가 |
|---|---|---|---|
| `at_loiter` | 입력 | 1 | `Guidance/WpManager` 출력 4 |
| `turns` | 입력 | 2 | `Guidance/TurnCount` 출력 1 |
| `req_turns` | 입력 | 3 | `Mission` 상자 안 Constant `Req` (값 `loiter_turns`) |
| `at_end` | 입력 | 4 | `Guidance/WpManager` 출력 5 |
| `mode` | 출력 | 1 | → `ModeSwitch` (바로), → `ModeDly` → `WpManager`·`TurnCount` (한 스텝 뒤) |

- 차트 설정: 동작 언어 **MATLAB**, 갱신 방식 **Discrete**, 샘플 시간 `Ts_ctrl` (0.05 s)
- 차트 경로: `W08_0_offline/Mission/MissionFSM`

## Stateflow 가 차트를 실행하는 규칙

| 규칙 | 내용 | 이 차트에서 |
|---|---|---|
| 실행 주기 | 샘플마다 한 번 깨어나 전이 조건을 검사함 | 0.05 s 마다 |
| 한 스텝에 전이 하나 | 전이가 일어나면 그 스텝은 끝남. 새 상태의 전이 조건은 다음 스텝에 검사 | `Loiter → WpFollow` 직후 같은 스텝에 `Finish` 로 가지 않음 |
| **실행 순서** | 한 상태에서 나가는 전이가 둘 이상이면 **번호가 작은 쪽을 먼저** 검사하고, 먼저 참이 된 하나만 일어남. 번호는 차트의 전이 시작점 옆 `1`·`2` 로 보임 | `WpFollow` 의 두 전이 — `→ Loiter` 가 **1**, `→ Finish` 가 **2** (차트에서 확인) |
| `en:` (entry) | 상태에 **들어갈 때** 한 번 실행 | `mode = 1;` 등 |
| `du:` (during) | 상태에 머무는 동안, 전이가 없는 스텝마다 실행 | 이 차트는 쓰지 않음 |
| `ex:` (exit) | 상태를 **떠날 때** 한 번 실행 | 이 차트는 쓰지 않음 |
| 출력 유지 | 출력 데이터는 다시 쓰기 전까지 **값을 유지**함 | 그래서 `en:` 한 줄로 충분함 |
| 차트 종류 | 기본값인 **Classic** 차트 (Moore·Mealy 아님) | `sflib/Chart` 로 만든 그대로 |

> ⚠ **`loiter_wp` 를 마지막 웨이포인트로 두지 않는다**
>
> - `loiter_wp = 5` 이면 WP5 에 닿는 스텝에 `at_loiter` 와 `at_end` 가 **동시에 1** 이 됨
> - `WpFollow` 에서 나가는 두 전이가 함께 참 → 실행 순서가 결과를 정함
> - 그래서 `W08_setup` 은 `loiter_wp` 를 **2 이상 $n-1$ 이하**로만 받음 (기본 경로에서는 2·3·4). 벗어나면 오류로 멈춤

> ℹ **연구실 UGV 예제와 같은 구조다**
>
> `UGV_Control_StateFlow_260529.slx` 의 미션 차트가
> `PrepareFirst1 → StartMission1 → StartMission2 → StopMission2` 네 상태이고,
> 로이터 종료 조건이 `number_of_turn >= required_number_of_turn` 임
> 이번 주차에 작성하는 것은 그중 **핵심 세 상태**만 추린 것임

## 1-3. 두 유도법칙을 갈아 끼우기

- 유도 블록 **두 개를 항상 같이 돌림.** 그리고 결과 중 하나를 고름

```
function [psi_ref, gate] = ModeSwitch(mode, psi_los, psi_lo)
if mode > 2.5
    psi_ref = psi_los;   gate = 0;    % 종료 – 속도 지령을 0 으로
elseif mode > 1.5
    psi_ref = psi_lo;    gate = 1;    % 로이터링 – 벡터필드
else
    psi_ref = psi_los;   gate = 1;    % 웨이포인트 – LOS
end
```

> ✎ **두 경로를 동시에 계산하는 이유**
> 안 쓰는 쪽을 꺼 두면 다시 켤 때 내부 상태(적분기·인덱스)가 엉킴
> 계산량이 문제될 만큼 무거운 블록도 아님
> **항상 돌리고 출력만 고르는 것**이 훨씬 안전함

---

## 1-4. 로이터 중에 웨이포인트가 넘어가면 안 된다

> ⚠ **이 주차에서 가장 놓치기 쉬운 지점**

- 웨이포인트 인덱스 `idx` 는 "다음 웨이포인트의 수락반경 $R$ 안에 들어오면" 한 칸 올라감
- 로이터 웨이포인트(WP3)에 닿는 스텝에 일어나는 일
  - `at_loiter = 1` → 차트가 `Loiter` 로 감
  - 같은 스텝에 `idx` 도 2 → 3 으로 올라감 (차트의 새 `mode` 는 한 스텝 뒤에 `WpManager` 에 닿기 때문, 1-5 절)
  - 그래서 로이터 중의 구간은 **WP3 → WP4**. 로이터가 끝나면 그 구간을 이어서 감
- 로이터 중에 배는 원을 두 바퀴 돎. **원이 WP4 의 수락반경을 지나가면** 막지 않은 인덱스는 WP4 에 "도착했다"고 판단함
  - `idx` 가 3 → 4 로 넘어감 → 로이터가 끝난 뒤 **WP4 를 건너뛰고** WP5 로 감
- 기본 경로에서는 WP4 가 원에서 28 m 이상 떨어져 있어 이 일이 생기지 않음. WP4 를 원 위로 옮기면 드러남

| **WP4 = (4, 47), 원 위** | **잠금 있음** | **잠금 없음** |
|---|---|---|
| 로이터 중 `idx` | 3 그대로 | **124.8 s 에 3 → 4** |
| 로이터 뒤 WP4 까지 최소 거리 | 0.2 m (들름) | **18.5 m (건너뜀)** |
| 로이터 뒤 주행 거리 | 88.2 m | 67.2 m |
| 임무 종료 | 305.9 s | 292.4 s |
| WP 구간 평균 $\lvert y_e \rvert$ | 2.865 m | 7.507 m (복귀 순간 $y_e \approx -37$ m) |

- 실측 조건: 기본 설정에서 `wp_north(4) = 4; wp_east(4) = 47;` 만 바꿈 (2026-09-18, 오프라인)
- 잠금이 없으면 임무가 **더 빨리 끝남** — 들러야 할 점 하나를 조용히 빼먹었기 때문. 성공처럼 보이는 실패

> **5주차는 "경로 방향" 판정이 기본인데 여기는 왜 수락 원인가**
>
> - 5주차 1-8절: 수락 원 판정은 **조류에 밀리면 걸릴 수 있음**
> - 이 주차는 아래 두 이유로 원을 그대로 씀
> - **조류가 없음.** 조류가 없으면 두 판정의 결과가 완전히 같음 (5주차 1-8절 표, $R = 5$ m · 조류 0 의 두 줄)
> - **로이터 진입은 "그 점에 왔다" 는 뜻임.** 경로 방향 판정은 옆으로 멀리 떨어져 있어도 구간 끝에 닿았다고 봄. 원을 그리며 돌 중심에 다가왔는지를 묻는 데는 거리가 더 맞는 질문임
>
> 조류를 넣는 과제로 확장할 때
>
> - `WpManager` 안의 `reached = d_next <= R;` 한 줄을 5주차 `GuidanceLaw` 처럼 경로 방향 판정 $d - x_e < R$ 로 바꿈
> - 또는 $R$ 을 넉넉히 둠

## 해결 — 인덱스 전진을 모드로 잠근다

```
% 웨이포인트 모드일 때만 인덱스를 올린다
if (mode < 1.5) && reached && (k < n-1)
    idx = k + 1;
else
    idx = k;
end
```

## 로이터 중심은 고정한다

- 로이터 중심을 "현재 웨이포인트"로 잡으면, 인덱스가 한 칸이라도 움직이는 순간 **원이 통째로 이동**함
- 그래서 중심을 **설정 시점에 상수로 고정**함

```
loiter_xc = wp_north(loiter_wp);
loiter_yc = wp_east(loiter_wp);
```

- 이렇게 두면 인덱스가 어떻게 되든 원은 그 자리에 있음

## 회전수도 모드에 묶는다

```
if mode < 1.5 || mode > 2.5
    turns = 0; acc_out = 0; armed = 0;   % 로이터 모드가 아니면 초기화
    return
end
```

- 로이터를 빠져나가면 카운터가 0 으로 돌아감
- 나중에 두 번째 로이터를 하더라도 **처음부터 다시 셈**

## 회전수는 원에 붙은 뒤부터 센다

- 로이터 모드에 들어와도 바로 세지 않음. 배와 중심 사이 거리 $r$ 이 목표 반경 $r_d$ (`loiter_radius`) 근처에 들어온 뒤부터 셈 (무장, `armed`)

$$|r - r_d| \le \text{tol} = 0.3\, r_d \quad (r_d = 18\ \text{m} \text{ 이면 } 5.4\ \text{m})$$

- 무장 뒤에는 중심에서 본 방위 $\theta$ 의 변화 $\Delta\theta$ 를 $(-\pi, \pi]$ 로 감아서 누적함 ($r \cdot \theta$ 의 뜻은 7주차 1-4절 경고표)

$$\text{turns} = \frac{|\sum \Delta\theta|}{2\pi}$$

- 진입 순간 배는 중심(= 로이터 웨이포인트) 에서 $R$ = 5 m 안에 있음 → 원으로 나가는 동안은 세지 않음
  - 그래서 로이터 체류 시간에는 중심에서 원까지 나가는 구간이 포함됨
  - 기본 설정 실측: 진입 74.6 s → **85.2 s 에 무장** ($r = 12.75$ m, $|r - r_d| = 5.25 \le 5.4$). 약 11 s 가 나가는 구간
- 무장 허용폭을 $0.3\,r_d$ 로 넓게 둔 이유: 정착 반경 오차가 $p_c v \tau_{\text{eff}}$ 만큼 남음 (7주차 1-8절). $p_c = 1$ 이면 약 7 m 라 좁은 폭으로는 **영영 무장하지 못함** (7주차 C-1 표)
- 허용폭은 7주차 `TurnCount` 와 같은 $0.3\,r_d$

---

## 1-5. 대수 루프 — 그리고 끊는 법

**✕ 이대로 연결하면 모델이 컴파일되지 않는다**

- 신호가 도는 모양을 보면

```
MissionFSM ──mode──→ WpManager ──at_loiter (입력 1)──┐
           │                    └─at_end    (입력 4)──┤
           └──────────→ TurnCount ──turns    (입력 2)──┤
                                                       └──→ MissionFSM
```

- **한 스텝 안에서** FSM 의 출력이 자기 입력으로 돌아옴
- Simulink 는 이것을 **대수 루프(algebraic loop)** 라고 부르고 거부함

```
'W08_0_offline/MissionFSM' 또는 이를 참조하는 모델은 대수 루프에서 지원되지 않습니다.
'W08_0_offline/MissionFSM'의 입력 포트(1, 2, 4)가 루프에 연결되어 있습니다.
```

### 끊는 법 — 단위지연

- `mode` 를 **Unit Delay** 한 개 거쳐서 되먹임

```
MissionFSM ──→ [ 1/z ] ──mode──→ WpManager, TurnCount
```

- `WpManager` 와 `TurnCount` 는 **한 스텝 전의 mode** 를 봄
- 제어 주기가 0.05 s 이므로 실질적인 차이가 없음

| 누가 | 보는 `mode` | 결과 |
|---|---|---|
| `ModeSwitch` | 차트 출력 그대로 (지연 없음) | 전이한 그 스텝에 유도법칙이 바뀜 |
| `WpManager` · `TurnCount` | `ModeDly` 를 거친 값 (0.05 s 전) | 인덱스 잠금·회전수 초기화가 한 스텝 늦게 걸림 |
| 로그 `log_mode` (그림·출력의 전이 시각) | `ModeDly` 뒤의 태그 `mode` | 찍히는 전이 시각이 실제 전이보다 **한 스텝 (0.05 s) 늦음** |

- `at_loiter` 는 **한 샘플 동안만 1** 인 펄스임
  - 도착한 그 스텝에 인덱스가 다음 구간으로 넘어가 $k + 1 = k_{\text{lo}}$ 가 곧바로 거짓이 되기 때문
  - 차트가 `Ts_ctrl` 로 같은 스텝에 돌기 때문에 이 펄스를 놓치지 않음 — 차트 샘플 시간은 `Ts_ctrl` 그대로 둘 것

**★ 직접 통과 블록만으로 닫힌 루프에는 상태가 하나 필요하다**

- 상태 = 한 스텝 전 값을 기억하는 블록 (Unit Delay) 또는 적분기
- 5주차의 웨이포인트 인덱스, 7주차의 회전수 누적은 Unit Delay 로 끊음
- 부록 A1 M절 SB13 은 차트 출력이 적분기 `Dist` 를 거쳐 돌아옴 → Unit Delay 없이도 대수 루프가 아님
- **"자기 출력을 같은 스텝의 자기 입력으로 쓸 수 없다"** — 이산 시스템의 기본 규칙

---

## 1-6. 로이터를 마치고 나면 배는 경로 밖에 있다

- 로이터 반경이 18 m 이면, 로이터를 끝낸 배는 로이터 중심(= WP3)에서 **약 18 m 떨어진 원 위**에 있음
- 다음 구간 WP3 → WP4 는 그 중심에서 시작함 → 배는 **경로 밖에서** 출발
- 경로 오차 $y_e$ 의 크기는 원 위 **어느 지점에서 빠져나오는가**에 따라 0 ~ 18 m
  - 원이 구간 직선과 만나는 두 점 부근에서 나오면 0 에 가깝고, 직선에서 가장 먼 점에서 나오면 18 m
- 기본 설정 실측: 로이터 종료 247.9 s 에 $y_e = -7.66$ m, 복귀 뒤 최대 $|y_e|$ 도 **7.66 m** (빠져나온 순간이 최대 — 그 뒤로는 7.59 m 까지 줄어듦)
- 이때 **LOS 가 다시 경로로 끌어옴**

**ℹ 5주차 실습이 여기서 쓰인다**

5주차에서 배를 일부러 20 m 벗어난 곳에서 출발시킨 이유가 이것임
"경로 밖에서 시작해 경로로 붙는 것"이 LOS 의 본래 역할이고,
미션 중에 그 상황이 **실제로 발생함**

---

## 1-7. 5·7주차에서 이름과 뜻이 바뀐 것

**⚠ 같은 이름이 다른 뜻으로 쓰이는 곳이 있다**

5·7주차 설정 파일을 보며 옮겨 올 때 아래 표를 먼저 확인할 것.

| 5·7주차 | 8주차 | 달라진 점 |
|---|---|---|
| 5주차 `GuidanceLaw` 출력 `psi_ref` | `WpManager` 출력 `psi_los` | LOS 지령에 새 이름. `psi_ref` 는 이제 `ModeSwitch` 가 고른 최종 지령 |
| 5주차 `mode` (1 = atan2, 2 = LOS) | `mode` (1 = WpFollow, 2 = Loiter, 3 = Finish) | **같은 이름, 다른 뜻.** 8주차 웨이포인트 유도는 LOS 하나뿐이고, `mode` 는 미션 상태 |
| 5주차 `wp_idx` | `idx` | 이름만 바뀜. 구간 시작 인덱스라는 뜻은 같음 |
| 5주차 `sw_mode` (기본 1 = 경로 방향) | 없음 | 8주차는 모든 구간을 **수락 원** $d_{k+1} \le R$ 로 판정 (1-4절 note) |
| 5주차 `gate` — `GuidanceLaw` 가 냄 | `gate` — `ModeSwitch` 가 냄 | `Finish` 상태에서 0 |
| 7주차 `r_d` | `loiter_radius` | 이름만 바뀜 |
| 7주차 중심 `center_north` · `center_east` | `loiter_xc` · `loiter_yc` | 지정한 웨이포인트 좌표로 자동 설정 |
| 7주차 `required_turns` | 설정 `loiter_turns` , 차트 입력 `req_turns` | 7주차는 0 이면 **멈추지 않고 계속 돎.** 8주차는 0 이면 조건 $\mathbf{turns} \ge 0$ 이 곧바로 참 → 다음 스텝에 `WpFollow` 로 돌아가 **사실상 로이터를 건너뜀** |

## 1-8. 미션 끝에 접안을 붙인다 — 유도를 끊고 위치제어가 받는다

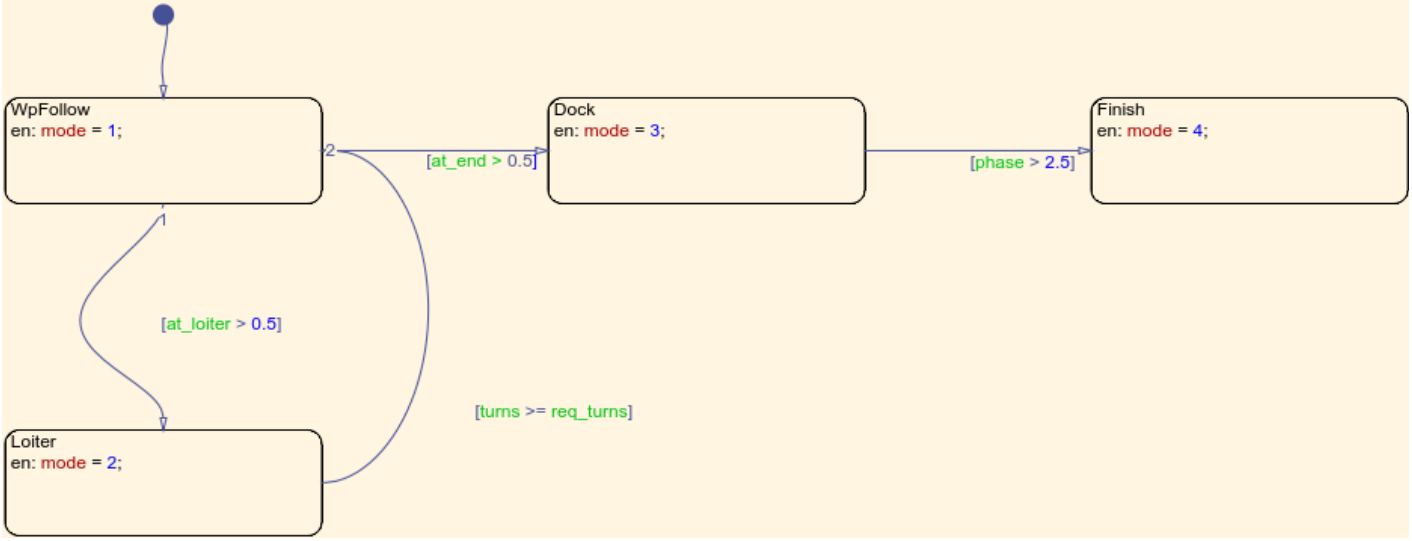
> ℹ **여기부터는 실습 E 의 내용임**
> - 1-1 ~ 1-7 절과 실습 A ~ D 는 상태 **셋**짜리 미션임. 그대로 둠
> - 본 절과 실습 E 는 그 끝에 네 번째 상태를 더한 **별도 모델** `W08_2_dock` 을 다룸
> - 설정 파일도 따로임 ( `W08_dock_setup.m` ). 실습 A 를 돌린 뒤 실습 E 를 돌려도 A 의 결과가 바뀌지 않음

### 유도법칙으로는 배를 세울 수 없다

| 유도법칙 | 내는 것 | 접안에 모자란 점 |
|---|---|---|
| LOS (5주차) | 선수각 지령 + **고정** 전진속도 $u_{\mathrm{ref}}$ | 세울 방법이 없음 — 속도 지령이 항상 1.5 m/s |
| 벡터필드 (7주차) | 선수각 지령 + **고정** 전진속도 | 같음. 원 위를 계속 돎 |
| 위치제어 (6주차) | 위치 오차 → 전후·횡 **속도 지령** | 목표점에 가까워질수록 지령이 스스로 0 으로 줄어듦 |

- 유도는 **방향**을 내고, 위치제어는 **자리**를 냄
- 선석은 점 하나임 — 방향만으로는 그 점에 설 수 없음
- 따라서 마지막 구간은 유도를 끊고 **6주차 위치제어**가 그대로 받음
  - 외부루프(위치 P) · 내부루프(서지·스웨이 PI, 선수각 P–D) · 3추진기 추력배분 — 구조와 게인 모두 6주차 것임. 새로 설계한 것은 없음

## 상태 넷 — `Dock` 이 끼어든다




```
        ●
        ↓
  ┌──────────┐  [at_end > 0.5]  ┌──────────┐  [phase > 2.5]  ┌──────────┐
  │ WpFollow │ ─────────────→   │   Dock   │ ─────────────→  │  Finish  │
  │ mode = 1 │                  │ mode = 3 │                 │ mode = 4 │
  └──────────┘                  └──────────┘                 └──────────┘
     ↓    ↑
     │    │ [turns >= req_turns]
[at_loiter > 0.5]
     ↓    │
  ┌──────────┐
  │  Loiter  │
  │ mode = 2 │
  └──────────┘
```

| 상태 | `mode` | 선수각 지령 | 전진속도 | 달라진 점 |
|---|---|---|---|---|
| `WpFollow` | 1 | LOS | $u_{\text{ref}}$ | 실습 A 와 같음 |
| `Loiter` | 2 | 벡터필드 | $u_{\text{ref}}$ | 실습 A 와 같음 |
| **`Dock`** | **3** | 부두 자세 | **위치제어가 냄** | **새로 생김** |
| `Finish` | 4 | 부두 자세 | 위치제어가 냄 | 실습 A 에서는 `mode = 3` 이었고 **정지**였음 |

- 앞 세 상태의 동작과 전이 조건은 실습 A 와 **글자 하나까지 같음**
- 실습 A 의 `WpFollow → Finish` 자리에 `Dock` 이 들어옴. 전이 조건 `at_end > 0.5` 도 그대로임
- 차트 입력이 하나 늘어 **5개** 가 됨 — `at_loiter` · `turns` · `req_turns` · `at_end` · **`phase`**
- 차트 경로: `W08_2_dock/Mission/MissionFSM`

> ⚠ **`Finish` 는 "멈춘다" 가 아니라 "선석을 지킨다" 이다**
>
> - 실습 A 의 `Finish` 는 모든 지령을 0 으로 끊었음 (배가 관성으로 미끄러지다 섬)
> - 실습 E 의 `Finish` 는 **위치제어를 그대로 살려 둠.** 기준점이 선석에 있으므로 위치 오차가 0 에 가깝고, 지령도 저절로 0 이 됨
> - 바람·파랑이 밀면 다시 지령이 생겨 제자리로 돌아옴 — 이것이 동적위치제어(6주차)의 본래 역할임

## 모드 선택은 곱셈 두 개와 덧셈 하나로 한다

- `ModeSwitch` 가 상태 번호를 받아 **문(gate)** 둘을 냄

$$u_{\text{cmd}} = \underbrace{K_{p,x}\, e_x}_{\text{위치제어}} \cdot g_p \;+\; \underbrace{u_{\text{ref}}}_{\text{유도}} \cdot g_u$$

| `mode` | $g_p$ | $g_u$ | 결과 $u_{\text{cmd}}$ |
|---|---|---|---|
| 1 `WpFollow` | 0 | 1 | $u_{\text{ref}}$ — 유도가 냄 |
| 2 `Loiter` | 0 | 1 | $u_{\text{ref}}$ — 유도가 냄 |
| 3 `Dock` | 1 | 0 | $K_{p,x}e_x$ — 위치제어가 냄 |
| 4 `Finish` | 1 | 0 | $K_{p,x}e_x$ — 위치제어가 냄 |

- 제어기 안에 `if` 를 두 번 쓰지 않음. **판단은 상태기계 한 곳에만 있음**
- 선수각 고리는 **어느 모드에서도 끊지 않음.** `Finish` 에서도 선수각을 부두와 나란히 붙잡아야 접안 자세가 풀리지 않음

## 접안은 세 단계로 나눈다 — `BerthRef`

- 위치제어는 "따라갈 점 하나" 를 받아야 함. 그 점을 내는 것이 `BerthRef` 임 (6주차 `TaskRef` 의 도킹 가지를 상태기계에 맞게 옮긴 것)

| `phase` | 기준점 $(x_{\text{ref}}, y_{\text{ref}})$ | 선수각 지령 | 다음 단계로 넘어가는 조건 |
|---|---|---|---|
| 0 접근점으로 | 접근점 $(-20,\ 90)$ | 접근점을 바라봄 | 접근점까지 거리 < `R_dock` = 4 m |
| 1 선수각 맞추기 | 접근점 (묶어 둠) | 부두 자세 180° | 자세오차 < 5° **그리고** $\lvert u\rvert$ < 0.20 m/s |
| 2 순수 옆걸음 | **선석** $(-20,\ 75)$ | 부두 자세 180° | 선석까지 거리 < `R_stop` = 1.5 m |
| 3 접안 완료 | 선석 | 부두 자세 180° | (마지막 단계. 이 값이 `Dock → Finish` 조건) |

- 단계는 한 번 오르면 내려가지 않는 **걸쇠**임. 자기 출력을 단위지연으로 되받음 ( `PhaseDly` )
  - 1-5 절의 대수 루프와 같은 구조임 — `phase` 가 상태기계로 가고, 상태기계가 낸 `mode` 가 다시 `BerthRef` 로 옴

> ★ **옆걸음은 배가 멈춘 뒤 시작한다**
>
> - `phase 1 → 2` 조건에 $\lvert u\rvert$ < 0.20 m/s 가 붙어 있음
> - 전후 속력이 남은 채 횡 지령을 주면 앞으로 미끄러지며 옆으로 감 — 궤적이 비스듬해지고 **순수 옆걸음이 아님**
> - 자세만 맞추고 바로 옆걸음을 시작하면 베이 입구 모서리에 선수가 걸림

## 왜 순수 옆걸음이 되는가

- 접근점은 선석에서 **부두와 직각인 방향**으로 `L_app` = 15 m 떨어진 점임

$$(x_a,\ y_a) = \left(x_d - s_d L_a \sin\psi_d,\ \ y_d + s_d L_a \cos\psi_d\right)$$

- `phase 1` 에서 선수각을 이미 부두 자세 $\psi_d$ 에 맞춰 두었으므로, 접근점과 선석을 잇는 선은 **선체 $y$ 축(횡방향)과 일치**함
- 따라서 `phase 2` 에서 기준점이 선석으로 바뀌는 순간
  - 선체좌표 전후 오차 $e_x \approx 0$ → 전후 속도 지령 $u_{\text{cmd}} \approx 0$
  - 선체좌표 횡 오차 $e_y$ 만 15 m 남음 → 횡 속도 지령 $v_{\text{cmd}}$ 가 `v_max` = 0.6 m/s 로 포화

- 선수각 고리는 계속 살아 있어 $\psi$ 를 180° 에 붙잡음
- 결과: **선수각이 고정된 채 배가 옆으로만 이동함**
  - 실측 — 옆걸음 전 구간 최대 자세오차 **0.255°**, 최대 $|u|$ 0.192 m/s · 최대 $|v|$ 0.599 m/s

## 횡력은 선수추진기가 낸다 — 3추진기 추력배분 (6주차 그대로)

- 좌·우 추진기는 둘 다 **전후 방향**임. 이 둘만으로는 횡력 $Y$ 를 낼 수 없음
- 선수추진기 $F_B$ 가 횡력을 담당함. 그것이 만드는 요 모멘트를 좌·우가 뺌

$$F_B = \mathrm{sat}\big(Y,\ \pm F_{B,\max}\big), \qquad M = N - l_3 F_B$$

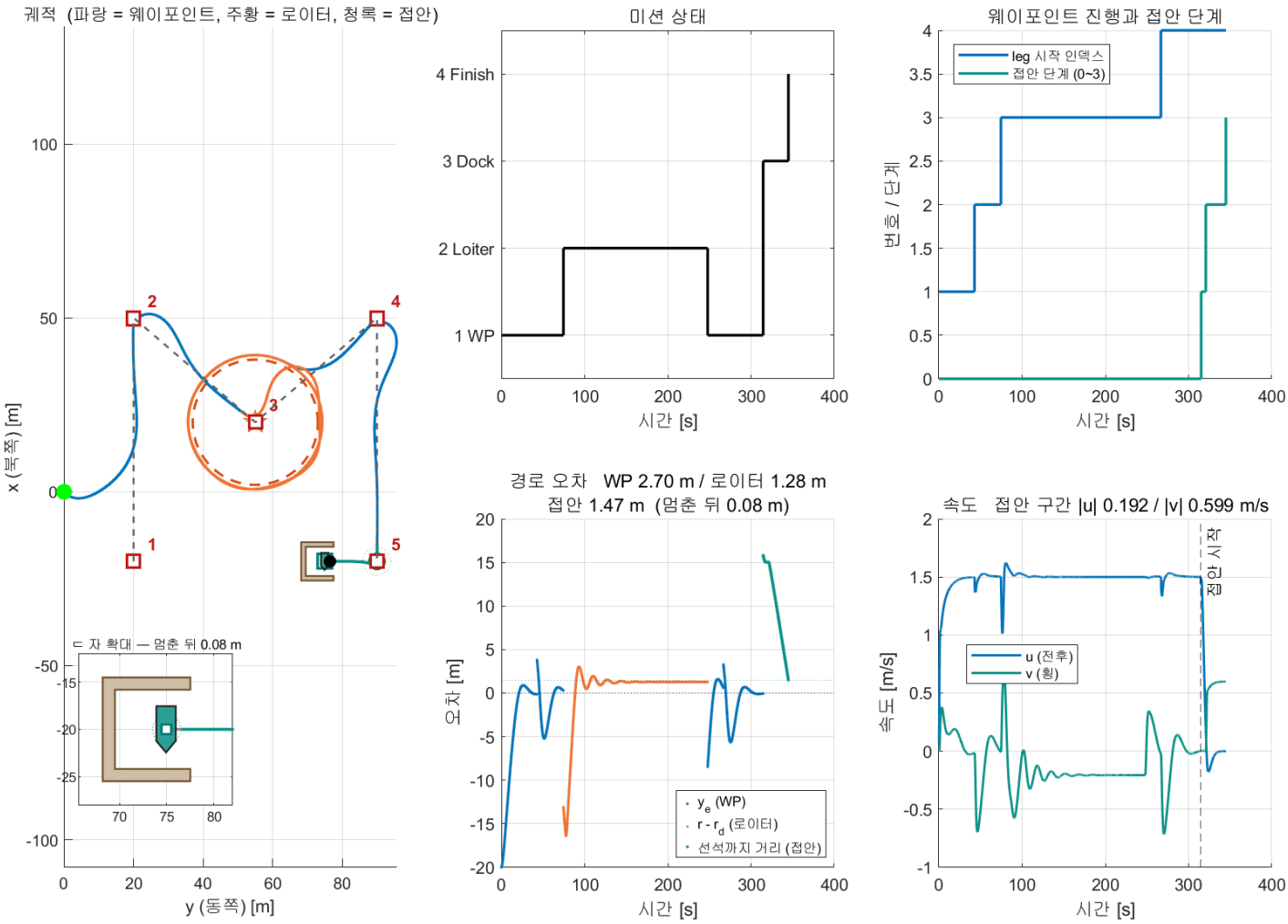
$$F_L = \frac{X}{2} + \frac{M}{2b}, \qquad F_R = \frac{X}{2} - \frac{M}{2b}$$

| 기호 | 뜻 | 값 |
|---|---|---|
| $b$ | 좌우 추진기 간 반폭 | 1.027 m |
| $l_3$ | 선수추진기의 선체중심 앞쪽 거리 | 0.75 m |
| $F_{\max}$ | 좌·우 추진기 한계 | 500 N |
| $F_{B,\max}$ | 선수추진기 한계 | 250 N |

- $X$ · $Y$ · $N$ 은 내부루프가 낸 전후력 · 횡력 · 요 모멘트임
- $N$ 은 **요 모멘트에만** 쓰는 기호임. 북쪽 좌표는 $x$ 임 (코드 `x_n`)

> ⚠ **선수추진기가 없는 WAM-V 로는 돌아가지 않는다**
>
> - VRX 기본 WAM-V 는 좌·우 추진기 둘뿐임. 횡력을 낼 수 없어 옆걸음이 되지 않음
> - 실습 E 의 VRX 검증은 6주차에서 쓴 **선수추진기 달린 WAM-V** (`W06_vrx_lite/wamv_bow_lite.urdf`) 로 함 (실습 E-4)

## ㄷ 자 도킹 스테이션 — 치수는 선체에서 나온다




- 모델이 쓰는 선체 — **반길이 2.45 m, 반폭 1.03 m** (실시간 화면·결과 그림과 같은 실척 다각형)
- ㄷ 자 치수는 `W08_dock_patch.m` 한 곳에 있음

| 기호 | 뜻 | 값 | 근거 |
|---|---|---|---|
| $H$ | 입구 반폭 | 4.2 m | 선체 반길이 2.45 m → 양옆 1.75 m 여유 |
| $K$ | 베이 깊이 | 5.5 m | 선체가 들어가 서는 자리 |
| $L_w$ | 날개 길이 | 2.5 m | 7 m 로 두면 들어가는 선체를 스침 (6주차 실측, 0.022 m 차이) |
| $W$ | 벽 두께 | 1.3 m | 그림에서 벽이 선으로 보이지 않게 하는 최소 두께 |

- 접안 완료 자세(선수각 180°)에서의 실측 여유

| 구간 | 베이 안 범위 [m] | 선체 범위 [m] | 여유 |
|---|---|---|---|
| 북–남 (입구와 나란한 방향) | $x \in [-24.20,\ -15.80]$ | $x \in [-22.45,\ -17.55]$ | 양쪽 각 **1.75 m** |
| 동–서 (들어가는 방향) | $y \in [69.50,\ 77.50]$ | $y \in [73.97,\ 76.03]$ | 안쪽 4.47 m · 입구쪽 **1.47 m** |

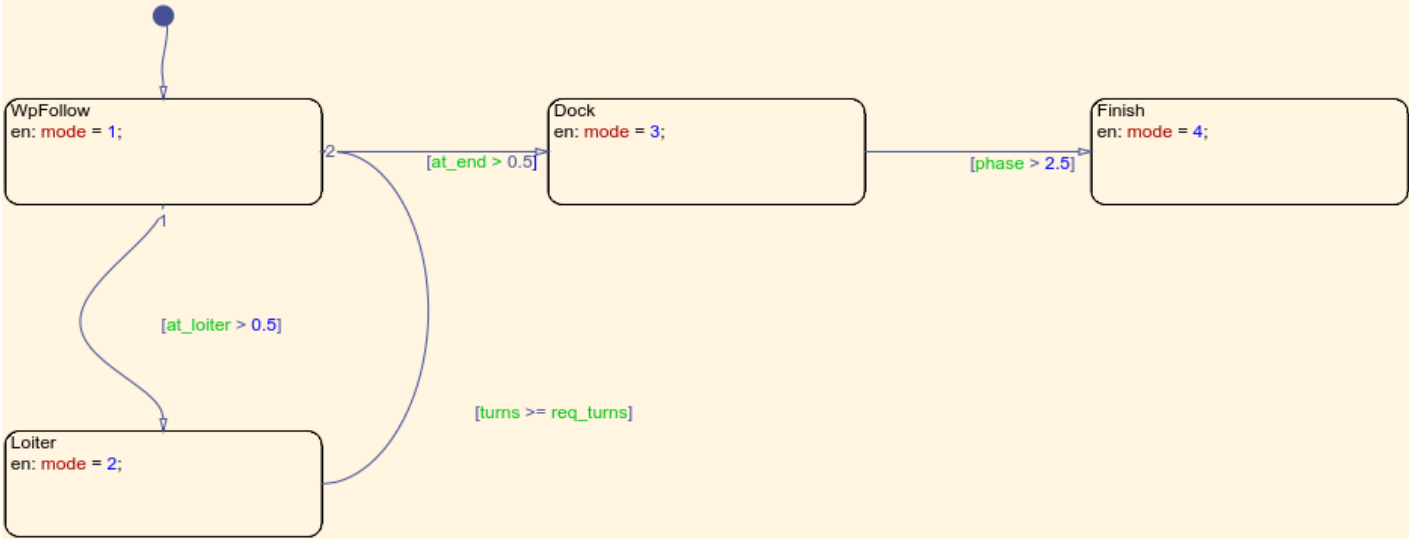
- 벽 바깥 남쪽 끝은 $x = -25.50$ — 안전 수역 남쪽 한계 $x \geq -28$ 까지 **2.50 m** 남음
- 웨이포인트 · 로이터 원 · 도킹 스테이션 · 접안 궤적이 모두 안전 수역 $x \in [-28,\ 60]$, $y \in [-18,\ 100]$ 안에 듦

---

## 1-9. Stateflow 차트가 실제로 어떻게 도는가 — 이번 주차의 핵심

> ★ **앞 절은 무엇을 만드는지, 이 절은 그것이 어떻게 도는지를 다룬다**
>
> - 1-1 ~ 1-8 절 — 상태 넷과 전이 넷을 **무엇으로** 정했는가
> - 1-9 절 — 그것이 **0.05 초마다 어떤 순서로** 실행되는가
> - 차트를 띄워 놓고 읽을 것 — `open_system('W08_2_dock/Mission/MissionFSM')`

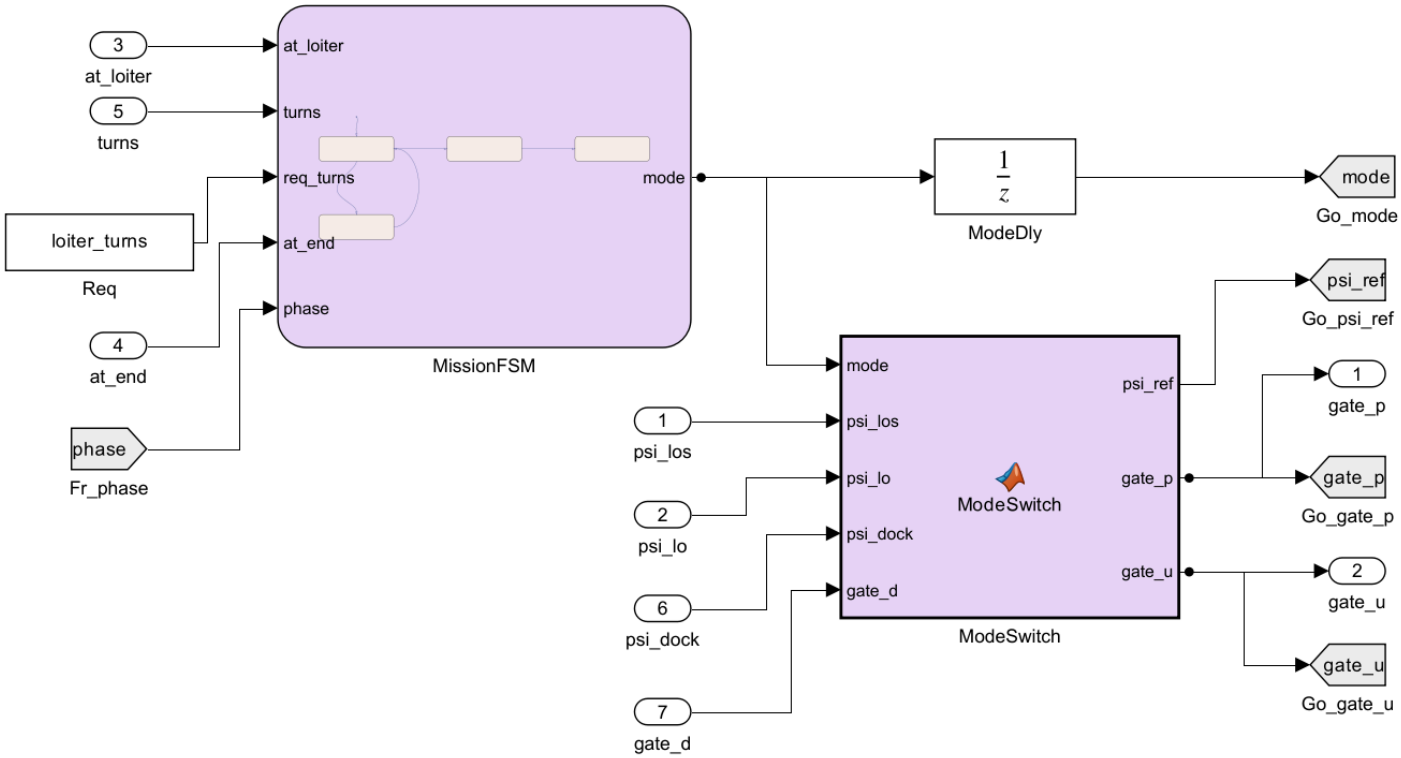
### 차트의 실제 화면




- 화면에 보이는 것의 뜻

| 화면 요소 | 뜻 |
|---|---|
| 굵은 점에서 나온 화살표 | **기본 전이** — 시뮬레이션이 시작되면 조건 없이 `WpFollow` 로 들어감 |
| 둥근 사각형 | **상태.** 첫 줄이 이름, 둘째 줄이 동작 |
| `en: mode = 1;` | 그 상태에 **들어갈 때 한 번** 실행되는 동작 ( `en` = entry) |
| 화살표에 붙은 `[ ... ]` | **전이 조건.** 참이면 그 화살표를 타고 옮겨 감 |
| 화살표 시작점 옆의 작은 숫자 | **검사 순서.** `WpFollow` 에서 `1` 이 로이터, `2` 가 접안 |

- 차트가 들어 있는 상자는 `Mission` 서브시스템임

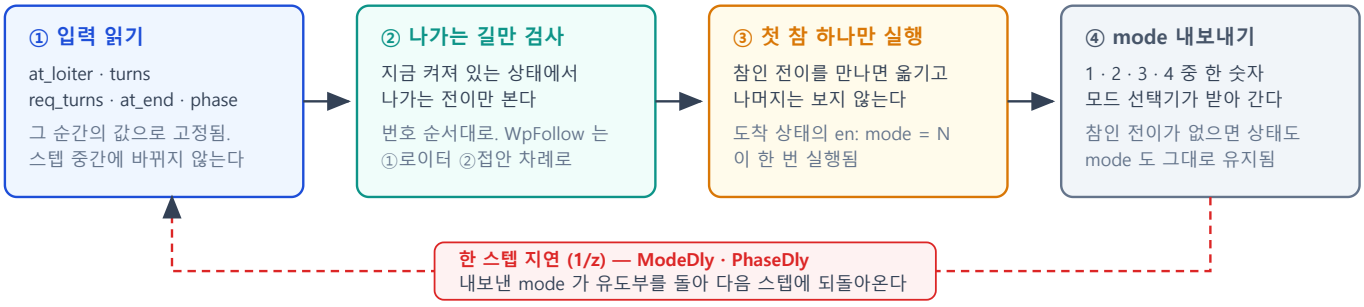



- 왼쪽 신호 다섯이 차트로 들어가고, 오른쪽으로 `mode` 하나가 나감
- 나온 `mode` 는 두 갈래로 감
  - `ModeSwitch` 로 **바로** — 어느 지령을 쓸지 고름
  - `ModeDly` ( `1/z` ) 를 거쳐 **한 스텝 뒤** — 유도부로 되돌아감

## 한 스텝에 일어나는 일 — 0.05 초마다 네 가지

**0.05 초마다 반복되는 한 스텝 — Stateflow 차트가 하는 일**

차트는 연속 블록이 아니다. 시계가 똑딱일 때만 깨어나 아래 네 가지를 순서대로 한 번 한다




지연이 없으면 같은 스텝 안에서 mode 가 자기 자신을 참조하게 되어 대수 루프가 된다 — Simulink 가 풀지 못한다
그래서 상태가 바뀐 결과는 0.05 초 뒤 유도부에 닿는다. 15 분짜리 미션에서는 보이지 않는 지연이다

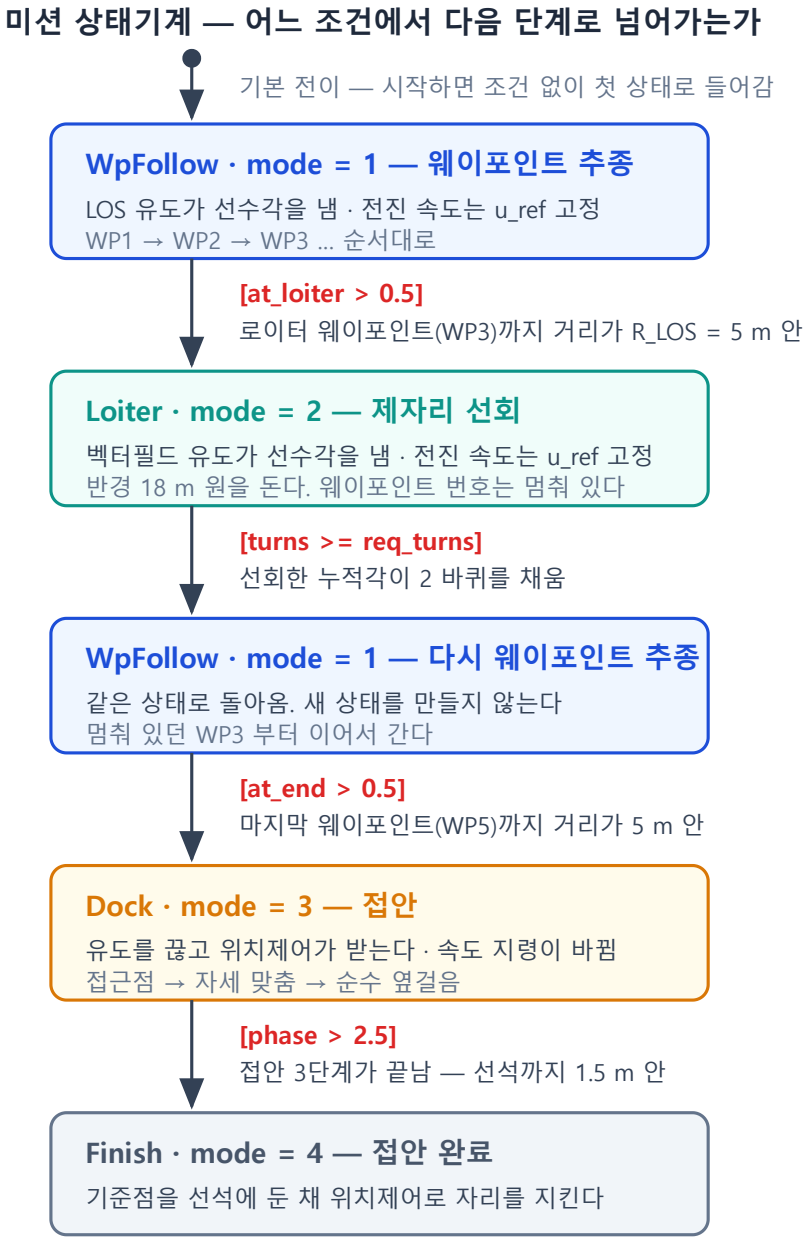
1. **입력을 읽음** — `at_loiter` , `turns` , `req_turns` , `at_end` , `phase` 다섯 개를 그 순간의 값으로 고정함
2. **지금 켜져 있는 상태에서 나가는 전이만 검사함** — 번호가 작은 것부터 차례로
3. **참인 전이를 처음 만나면 그 하나만 실행함** — 상태를 옮기고 도착 상태의 `en: mode = N` 을 한 번 실행함
4. `mode` **를 내보냄** — 참인 전이가 하나도 없으면 상태도 `mode` 도 그대로 유지됨

> **ℹ 차트는 연속 블록이 아니다**
>
> - 설정이 **Discrete**, 샘플 시간이 `Ts_ctrl` = 0.05 s 임
> - 시계가 똑딱일 때만 깨어나 위 네 가지를 한 번 함. 그 사이에는 아무 일도 하지 않음
> - 상태가 바뀌는 시각이 늘 0.05 초의 배수로 찍히는 이유임

- 비유 — 0.05 초마다 한 번 눈을 뜨는 당직자

| 당직자 | 차트 |
|---|---|
| 계기판 숫자 다섯을 읽음 | 입력 다섯을 읽음 |
| 지금 맡은 임무의 교대 조건만 확인함 | 켜진 상태에서 나가는 전이만 검사함 |
| 조건이 맞으면 다음 임무로 넘기고 그 자리에서 끝냄 | 전이 하나만 실행하고 스텝을 끝냄 |
| 교대 내용을 칠판에 적어 둠 | `mode` 를 출력에 써 둠 |

## 네 번의 전이 — 무엇이 참이 되어야 넘어가는가




**조건을 만드는 곳**

at_loiter · at_end
WpManager (유도부)

turns
TurnCount (유도부)

phase
BerthRef (유도부)

req_turns
설정값 loiter_turns

상태기계는 조건을 만들지
않는다. 받아서 고를 뿐이다

| 전이 | 조건 | 말로 옮기면 | 조건을 만드는 블록 |
|---|---|---|---|
| (시작) → `WpFollow` | 없음 | 시뮬레이션이 시작됨 | — |
| `WpFollow` → `Loiter` | `at_loiter > 0.5` | 로이터 웨이포인트(WP3)까지 거리가 `R_LOS` = 5 m 안 | `Guidance/WpManager` |
| `Loiter` → `WpFollow` | `turns >= req_turns` | 선회한 누적각이 2 바퀴를 채움 | `Guidance/TurnCount` |
| `WpFollow` → `Dock` | `at_end > 0.5` | 마지막 웨이포인트(WP5)까지 거리가 5 m 안 | `Guidance/WpManager` |
| `Dock` → `Finish` | `phase > 2.5` | 접안 3단계가 끝남 — 선석까지 1.5 m 안 | `Guidance/BerthRef` |

> ★ **상태기계는 조건을 만들지 않는다**
> - 다섯 입력은 모두 유도부의 MATLAB Function 이 계산한 값임
> - 차트가 하는 일은 그 값을 보고 **지금이 몇 번째 단계인지 고르는 것** 하나뿐임
> - 거리 계산·각도 누적·접안 단계 판정은 전부 차트 **바깥**에 있음

## 실제로 잰 전이 시각 (오프라인, 전 구간 900 s)

| 시각 [s] | 일어난 일 | 참이 된 조건 |
|---|---|---|
| 0.00 | `WpFollow` 로 들어감 ( `mode` 1) | 기본 전이 |
| **74.60** | `Loiter` 로 ( `mode` 2) | `at_loiter > 0.5` |
| **247.85** | `WpFollow` 로 되돌아옴 ( `mode` 1) | `turns >= req_turns` ( `turns` = 2.000) |
| **314.55** | `Dock` 으로 ( `mode` 3) | `at_end > 0.5` |
| 315.15 | 접안 `phase` 0 → 1 | 접근점까지 거리 < `R_dock` = 4 m |
| 320.70 | 접안 `phase` 1 → 2 (옆걸음 시작) | $\lvert e_\psi \rvert < 5°$ 이고 $\lvert u \rvert < 0.20$ m/s |
| 344.85 | 접안 `phase` 2 → 3 | 선석까지 거리 < `R_stop` = 1.5 m |
| **344.90** | `Finish` 로 ( `mode` 4) | `phase > 2.5` |

- 상태별로 머문 시간 — `WpFollow` 141.3 s · `Loiter` 173.2 s · `Dock` 30.4 s · `Finish` 555.1 s

> ℹ **`Dock` 에 들어가자마자 `phase` 가 1 이 되는 이유**
> - 마지막 웨이포인트 WP5 `(x, y) = (-20, 90)` 와 접근점 `(-20.0, 90.0)` 이 **같은 점**임
> - `at_end` 가 참이 된 자리가 곧 접근점이므로 1단계(접근점으로 가기)에 할 일이 남지 않음
> - 그래서 `phase` 0 이 0.6 s 만 지속됨. 접근점을 WP5 와 다른 곳에 두면 이 구간이 길어짐

## 조건이 참이 된 뒤 한 스텝 늦게 넘어간다

- 위 표에서 `phase` 가 3 이 된 시각은 **344.85 s**, `Finish` 로 넘어간 시각은 **344.90 s** 임
- 차이 0.05 s — 정확히 한 스텝임
- 이유: 한 스텝 안에서 유도부( `BerthRef` )가 `phase` 를 새로 쓰는 시점이 차트가 입력을 읽은 **뒤**임
- 차트는 그 새 값을 **다음 스텝에** 봄

> ⚠ **한 스텝 지연은 고장이 아니라 설계다**
>
> - `mode` 는 유도부로 돌아가고, 유도부가 낸 신호는 다시 차트로 들어옴 — 고리임
> - 지연이 없으면 같은 스텝 안에서 `mode` 가 자기 자신을 참조하게 됨 (**대수 루프**, 1-5절)
> - `ModeDly` ( `1/z` ) 가 그 고리를 끊음. 대가는 0.05 s 의 지연 하나임
> - 15 분짜리 미션에서 0.05 s 는 궤적에 보이지 않음

## 전이 둘이 동시에 참이면

- `WpFollow` 에서 나가는 전이는 둘임 — `→ Loiter` 와 `→ Dock`
- 차트에서 실제로 조회한 검사 순서

| 전이 | 검사 순서 | 조건 |
|---|---|---|
| `WpFollow → Loiter` | **1** | `at_loiter > 0.5` |
| `WpFollow → Dock` | **2** | `at_end > 0.5` |

- 둘 다 참인 스텝이 생기면 **1 번이 이김** — 로이터로 감
- 그런 스텝은 `loiter_wp` 를 마지막 웨이포인트로 두었을 때만 생김 (1-2절 warning)
- `W08_dock_setup` 이 `loiter_wp` 를 $2 \le k_{\mathrm{lo}} \le n-1$ 로만 받아 애초에 막음

> ✎ **순서는 그린 순서로 정해진다**
>
> - 생성 스크립트 `build_w08_dock.m` 이 `Stateflow.Transition` 을 만드는 **순서**가 곧 검사 순서임
> - 차트를 손으로 고칠 때는 화살표 시작점 옆의 숫자를 확인할 것
> - 숫자를 바꾸려면 화살표를 **우클릭 → Execution Order** 에서 고름

## `mode` 가 제어로 가는 길 — 판단과 실행의 분리

```
MissionFSM ──mode──→ ModeSwitch ─┬── psi_ref   (선수각 지령: LOS / 벡터필드 / 접안 중 하나)
                                 ├── gate_u    (전진 속도를 u_ref 로 둘 것인가)
                                 └── gate_p    (전·횡 속도를 위치제어가 낼 것인가)
```

| `mode` | 상태 | `psi_ref` | `gate_u` | `gate_p` |
|---|---|---|---|---|
| 1 | `WpFollow` | `psi_los` | 1 | 0 |
| 2 | `Loiter` | `psi_lo` | 1 | 0 |
| 3 | `Dock` | `psi_dock` | 0 | 1 |
| 4 | `Finish` | `psi_dock` | 0 | 1 |

- 두 문은 **동시에 1 이 되지 않음** — 전 구간 18,001 스텝 가운데 동시에 1 인 스텝 **0 개** (실측)
- 그래서 `OuterLoop` 에서 두 가지를 더하면 그것이 곧 모드 선택이 됨

$$u_{\mathrm{cmd}} = K_{p,x}\, e_x\, g_p + u_{\mathrm{ref}}\, g_u$$

> ★ **모드를 고르는 `if` 문을 두 곳에 두지 않는다**
>
> - 판단은 `MissionFSM` **한 곳**에만 있음
> - 그 아래는 전부 `mode` 를 받아 쓰기만 함 — 유도부·제어부 어디에도 모드 판단이 없음
> - 미션을 바꾸려면 차트만 고치면 됨. 이것이 상태기계를 쓰는 이유임

### 차트가 하지 않는 일

| 하지 않는 일 | 어디서 하는가 |
|---|---|
| 거리·각도 계산 | `Guidance/WpManager`, `TurnCount`, `BerthRef` |
| 선수각·속도 지령 생성 | `Guidance` 의 LOS·벡터필드· `BerthRef` |
| 추력 계산 | `InnerLoop` 와 `Thrusters` |
| 웨이포인트 번호 전진 | `WpManager` ( `mode` 를 받아 잠금) |

- 차트의 출력은 **숫자 하나**( `mode` )뿐임
- 차트를 읽을 때는 "이 숫자가 언제 바뀌는가"만 보면 됨

---

# 2부 · 실습

## 실행하면 이런 결과가 나온다

**★ 아래는 기준 환경에서 실제로 받은 출력이다**

값이 크게 다르면 설정 파일을 먼저 확인함. 소수점 마지막 자리는 달라질 수 있음

```
cd('<배포 폴더>/W08_simulink')
W08_setup
```

- `W08_setup` 한 줄이 설정 → 모델 실행( `sim` ) → 그림·지표 출력( `W08_plot` )까지 함 ( `auto_run = 1` )
  - 따로 `sim` 을 다시 부를 필요 없음. 결과는 작업공간의 `out` 에 남음
- 정상 출력

```
W08 설정 완료
  웨이포인트 5개
  WP3 도착 시 로이터링 – 반경 18 m, 2 바퀴, 시계방향
  로이터 중심 (x,y) = (20, 55)
  목표 속도 1.5 m/s, 크랩 보상 1

오프라인 모델 실행 중  (auto_run = 0 으로 두면 실행하지 않는다)

===== WP -> 로이터(WP3, 2바퀴) -> WP =====
  로이터 진입              74.6 s
  로이터 종료             247.9 s   (2.00 바퀴)
  임무 종료               314.4 s
  WP 구간 평균 |y_e|      2.679 m
  로이터 평균 반경오차    1.282 m
```

| 읽는 법 | 뜻 |
|---|---|
| 진입 74.6 s → 종료 247.9 s | 로이터에 **173.3 초** 머물렀음. 앞부분은 중심에서 원으로 나가는 구간이라 회전수를 세지 않음 (1-4절) |
| (2.00 바퀴) | 정확히 2바퀴에서 빠져나왔음. 어긋나면 회전수 세기를 의심함 |
| 임무 종료 314.4 s | 로이터 뒤 남은 웨이포인트를 마저 돌았다는 뜻 |
| 반경오차 1.282 m | 크랩 보상 켬( `crab_comp = 1` ) → 7주차 1-8절 예측 $p_c\, v\, \tau_{\text{eff}} = 0.313 \times 1.5 \times 2.8 = 1.31$ m 와 일치. 7주차 2.136 m 는 보상 끔 |

**ℹ 반경오차 두 값은 조건과 계산 방식이 다르다**

- 7주차 2.136 m: `crab_comp = 0` , $r_d = 25$ m, 원에 도달한 뒤 평균
- 8주차 1.282 m: `crab_comp = 1` , $r_d = 18$ m, 로이터 표본의 앞 30 % 를 버리고 평균 ( `W08_plot.m` )

> ℹ **$p_c$ 는 반경 25 m 기준으로 정한 값이다**
> - 7주차 식 (28) $p_c = 4\zeta_c^2 v\tau_r / r_d$ 는 반경 $r_d$ 에 반비례
> - 8주차는 $r_d = 18$ m 인데 $p_c = 0.313$ ($r_d = 25$ m 로 계산한 값) 을 그대로 씀
> - $r_d = 18$ m 로 다시 계산하면 $p_c = 4 \times 1^2 \times 1.5 \times 1.31/18 = 0.437$
> - 0.313 을 그대로 두면 $\zeta_c = \frac{1}{2}\sqrt{p_c r_d/(v\tau_r)} \approx 0.85$ 로 1 보다 작아져 원에 붙는 과정이 조금 덜 감쇠됨. 대신 반경오차 $p_c v \tau_{\text{eff}}$ 는 작게 유지됨

> ✎ **미션 모델에서 가장 중요한 값은 상태가 바뀐 시각이다**
> 궤적 오차[m] 보다 전이 시각[s] 을 봄
> 오프라인과 VRX 를 비교할 때도 이 시각을 맞춤 → 궤적이-겹치면-이격은-오차가-아니라-지연이다

---

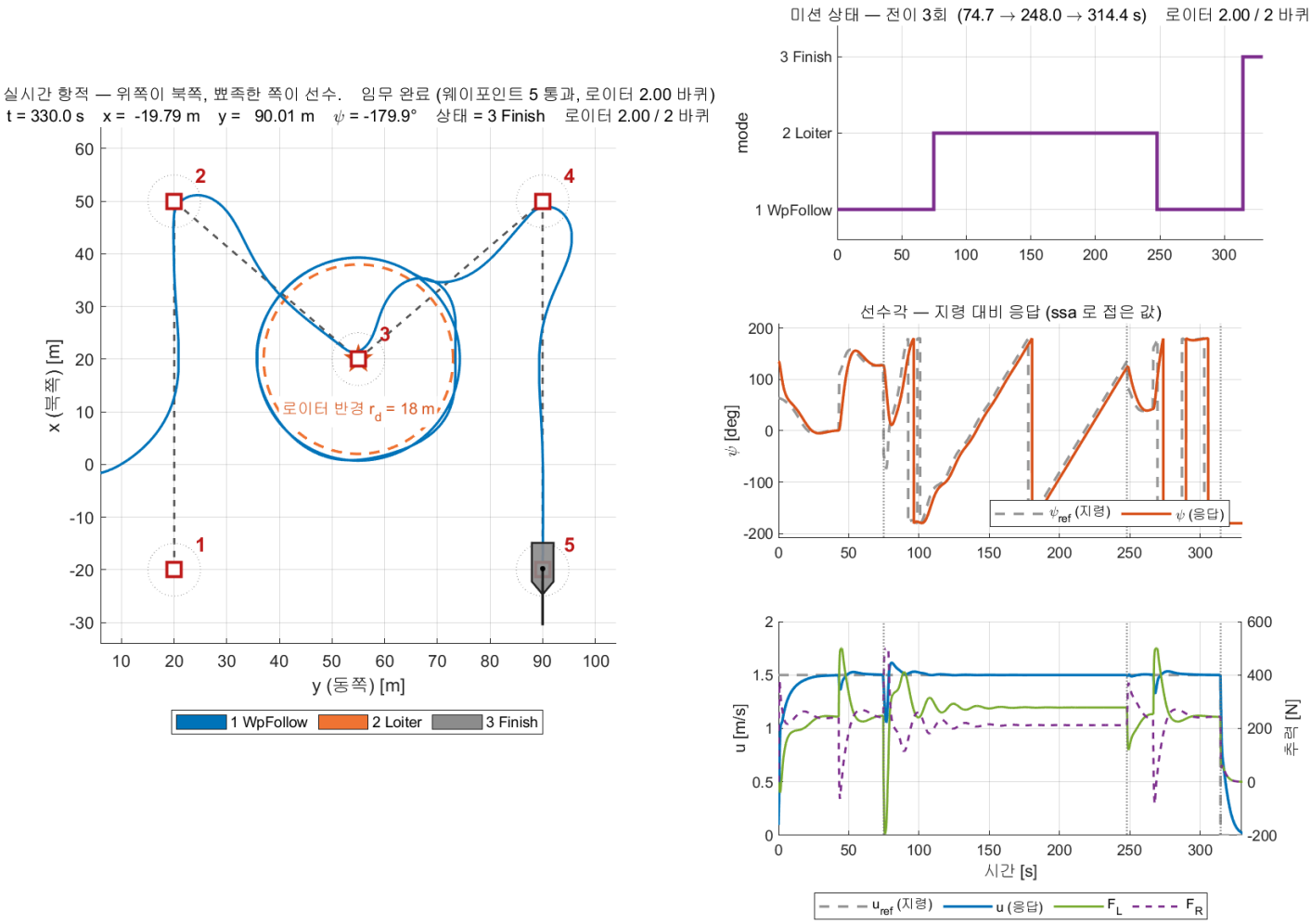
## A. 미션 한 번 돌려 보기 (20분)

```
cd('<배포 폴더>/W08_simulink')
W08_setup
```

- 정상 출력은 앞의 「실행하면 이런 결과가 나온다」 블록과 같음
- 화면에 창 세 개가 뜸 — 실시간 창, 성능 그림(2×3), 추력 그림

### 실시간 창 — 네 칸의 뜻

- 4·5·7주차와 **같은 네 칸 배치**임. 오른쪽 세 칸의 내용만 8주차 것으로 바뀜
- 7주차까지 화면이 답한 질문은 "시킨 대로 따라갔는가" 였음 (4주차 2-0-1, 5주차 A절)
- 8주차가 답해야 하는 질문은 하나 더 있음 — **지금 무엇을 할 차례이고, 언제 바뀌었는가**




- 조건 — `W08_0_offline`, 웨이포인트 5개, `loiter_wp = 3`, $r_d = 18$ m, 2바퀴, $p_c = 0.313$, `crab_comp = 1`, $\Delta = 10$ m, $R = 5$ m, $u_{\text{ref}} = 1.5$ m/s, $K_{p,\psi} = 400 \cdot K_{d,\psi} = 200$, $K_{p,u} = 300 \cdot K_{i,u} = 100$, 조류 0, 330 s 주행 (2026-09-25 실측)

| 자리 | 그리는 것 | 왜 그 자리에 있는가 |
|---|---|---|
| 왼쪽 큰 칸 | 항적 + 웨이포인트 · 수락반경 + **로이터 중심과 반경 원** + 선체 모양과 선수 방향 | 궤적선만으로는 뱃머리가 어디를 보는지 안 보임. **선체 색이 곧 지금 상태**이고, 그 뜻은 아래 범례에 있음 |
| 오른쪽 ① | **상태 타임라인** — `mode` 가 시간에 따라 바뀌는 모양을 계단으로 | **8주차의 핵심 지표.** 배 색은 "지금" 만 말하고 "언제" 를 말하지 못함. 미션 모델에서 가장 중요한 수치는 궤적 오차가 아니라 **전이 시각**임 (A 절 바로 앞의 콜아웃) |
| 오른쪽 ② | $\psi_{\text{ref}}$ (회색 점선) 와 $\psi$ (주황) 를 **겹쳐서** (deg, `ssa`) | 미션이 고른 지령을 헤딩 루프가 따라가는지 한 칸에서 읽힘 |
| 오른쪽 ③ | 왼쪽 축에 $u_{\text{ref}} \cdot u$, 오른쪽 축에 $F_L \cdot F_R$ | 그 응답을 무엇이 만들었는지 |

- 가로축은 `y (동쪽)`, 세로축은 `x (북쪽)` — **위쪽이 북쪽**이며 해도와 같은 방향임 (3주차 1-5)
- **배 색이 상태를 나타냄.** 왼쪽 칸 아래 범례가 같은 색을 상태 이름과 묶어 줌

| 색 | 상태 |
|---|---|
| 파랑 | `WpFollow` — 웨이포인트 항주 |
| 주황 | `Loiter` — 로이터링 |
| 회색 | `Finish` — 종료 |

- ② 의 점선은 모드에 따라 **다른 유도법칙이 낸 값**임 — `mode 1` 이면 `WpManager` 의 LOS, `mode 2` 면 `LoiterVF` 의 벡터필드. `ModeSwitch` 가 고른 뒤의 값을 받으므로 한 선에 둘이 섞여 있음. **전이 순간에 지령선이 툭 튀는 것이 정상임** — 법칙이 갈린 지점임
- ③ 이 `yyaxis` 로 축을 둘로 나눈 이유는 자릿수임. $u$ 는 1.x m/s, $F$ 는 수백 N 이라 한 축에 겹치면 $u$ 가 직선으로 보임
- ③ 의 지령선은 설정값 $u_{\mathrm{ref}}$ 가 아니라 $u_{\mathrm{ref}} \times \mathrm{gate}$ 임. `Finish` 로 들어가면 `gate` 가 0 이 되어 지령이 0 으로 떨어짐 (그림에서 314 s 부근)
- 오른쪽 ②·③ 의 **점선 세로선이 전이 시각**임. 세 칸이 같은 시간축을 쓰므로 위아래로 훑어 읽으면 됨
- 바퀴 수는 ① 의 제목에 있음. `TurnCount` 는 로이터에서 나오면 0 으로 되돌아가므로, 화면은 **그 실행에서 채운 최댓값**을 보여 줌 (0 으로 돌아간 값을 그대로 쓰면 캡처에 `0.00 바퀴` 가 남음)

**어디를 보면 미션이 돌고 있는가**

| 볼 곳 | 정상이면 | 아니면 |
|---|---|---|
| ① 의 계단 | `1 WpFollow` → `2 Loiter` → `1 WpFollow` → `3 Finish` 로 **세 번 바뀜.** 제목에 전이 시각 세 개가 찍힘 (74.6 → 247.9 → 314.4 s) | 2 로 안 올라가면 `at_loiter` 판정을, 1 로 안 내려오면 회전수 세기를 의심 (1-4절) |
| ① 제목의 바퀴 수 | **로이터 구간에서 늘어 2.00 에서 빠져나옴** | 2.00 을 못 채우고 나오면 전이 조건 `[turns >= req_turns]` 를 볼 것 |
| 왼쪽 칸의 배 색 | 파랑 → 주황 → 파랑 → 회색 순으로 **세 번 바뀜** | 주황이 안 나오면 WP3 수락반경에 못 닿은 것임 |
| 왼쪽 칸의 초록 원 | 수락반경에 닿을 때마다 다음 웨이포인트로 옮겨 가고, **복귀 후 WP4 를 다시 집음** | 로이터 뒤에도 표식이 안 옮겨 가면 `idx` 가 멈춘 것임 (로이터 중에는 일부러 멈춤 — 1-4절) |
| ② 의 두 선 | 응답선이 지령선에 **붙어서 같이 감.** 전이 시각의 세로선에서만 지령선이 툭 튐 | 붙지 않으면 $K_{p,\psi}$ 가 작거나 추력이 포화임 |
| ② 의 세로 점프 | $\pm 180^\circ$ 이음매를 지날 때만 생김 — `ssa` 로 접은 값이라 그럼. 고장이 아님 | — |
| ③ 의 지령선 | 회색(종료)이 되는 순간 **0 으로 떨어지고 $u$ 가 따라 내려옴** | 안 떨어지면 `gate` 가 `Finish` 에서 0 이 되는지 볼 것 |

| 설정 ( `W08_setup.m` ) | 뜻 |
|---|---|
| `animate = 1` | 0 으로 두면 그림을 그리지 않아 훨씬 빨리 돎 (330 s 주행 기준 29.4 s → 3.8 s, 2026-09-25 실측) |
| `animate_every = 0.25` | 몇 초마다 다시 그릴지. 키우면 빨라짐 |
| `boat_scale = 2` | 선체를 몇 배로 크게 그릴지. 1 이면 실제 크기(4.9 m) |

- 선체 모양은 5·7주차와 같음 — **선수는 세모, 선미는 네모**
- 빨간 사각형이 웨이포인트, 점선 원이 수락반경 $R$, 굵은 주황 점선 원이 로이터 궤도 $r_d$

**★ 오프라인 모델과 VRX 쌍둥이가 화면 함수 하나( `W08_animate.m` )를 같이 씀**

갈아 끼우는 것은 신호를 어디서 뽑았는지뿐임 — 운동방정식이냐, 참값 오도메트리냐
따라서 위 그림과 D-3 의 VRX 화면은 **같은 함수가 그린 같은 배치**이며 나란히 놓고 읽을 수 있음

**ℹ 모델 안에서는 포트 없는 회색 상자 `Animate` 하나임**

신호는 `From` 태그로 받으므로 **제어 신호 사슬에 선이 한 줄도 늘지 않음**
받는 태그는 `x_n` · `y_n` · `psi` · `mode` · `turns` · `psi_ref` · `u` · `gate` · `FL` · `FR` · `idx` 열한 개이며, 전부 **이미 있던 태그**임. 속도 지령의 곱 $u_{\mathrm{ref}} \times \mathrm{gate}$ 는 그리는 함수가 함
MATLAB Function 블록은 코드 생성 대상이라 기본적으로 그래픽 출력을 지원하지 않음.
`coder.extrinsic` 으로 선언하면 **컴파일하지 않고 MATLAB 함수를 그대로 부름**

```
coder.extrinsic('W08_animate');
```

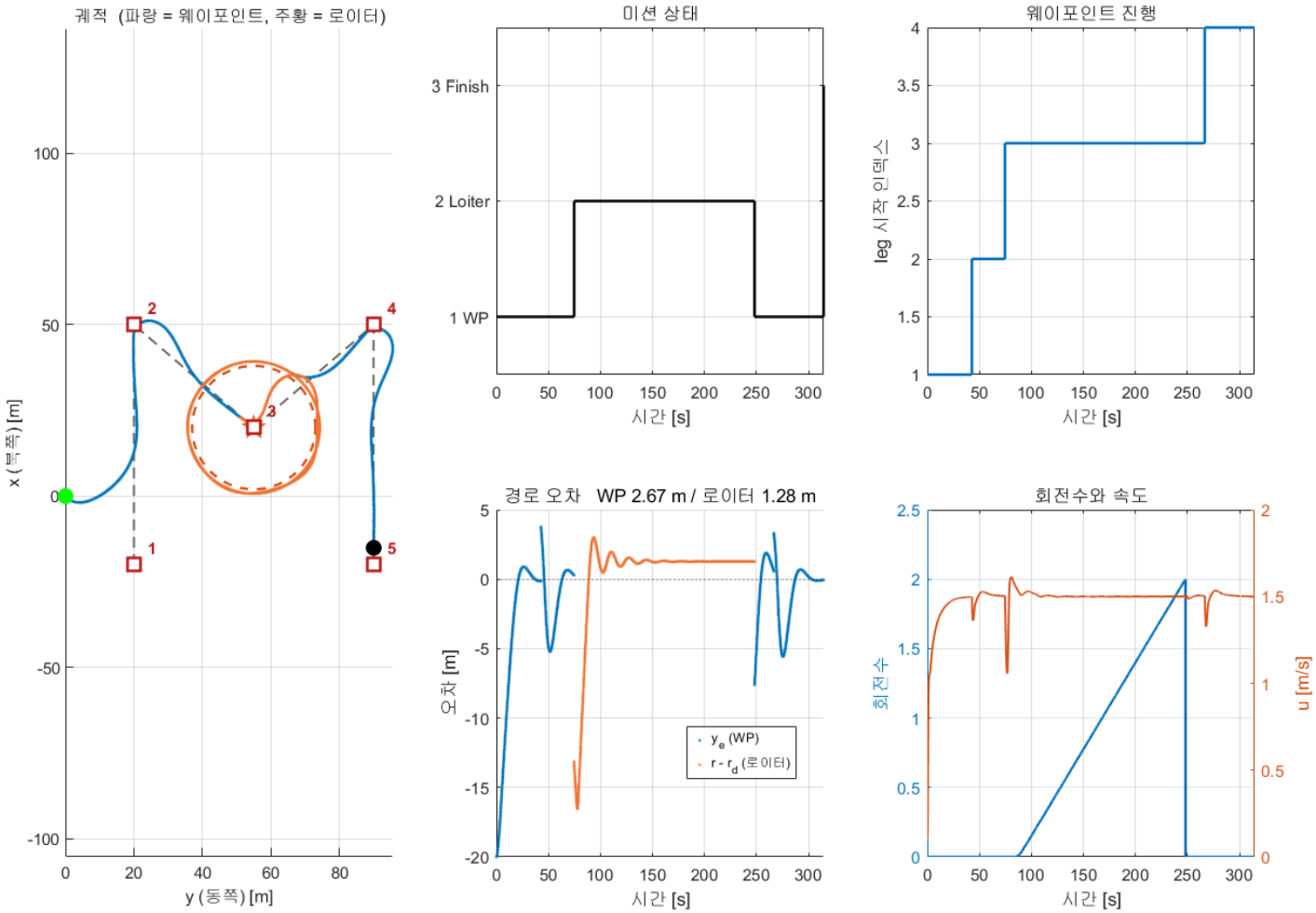
**✎ 이 화면을 그대로 PNG 로 남기려면**

```
W08_anim_capture('offline')              % img/W08_anim_offline.png
W08_anim_capture('vrx')                  % img/W08_anim_vrx.png   (VRX 기동 필요)
W08_anim_capture('offline','Stop',120)   % 120초에서 끊어 찍기
```

조건(게인 · 미션 설정 · 정지 시간 · 페이싱)과 **전이 시각·회전수**를 구조체로 함께 돌려주므로
그림과 조건이 어긋나지 않음. 전이가 세 번에 못 미치면 경고를 냄

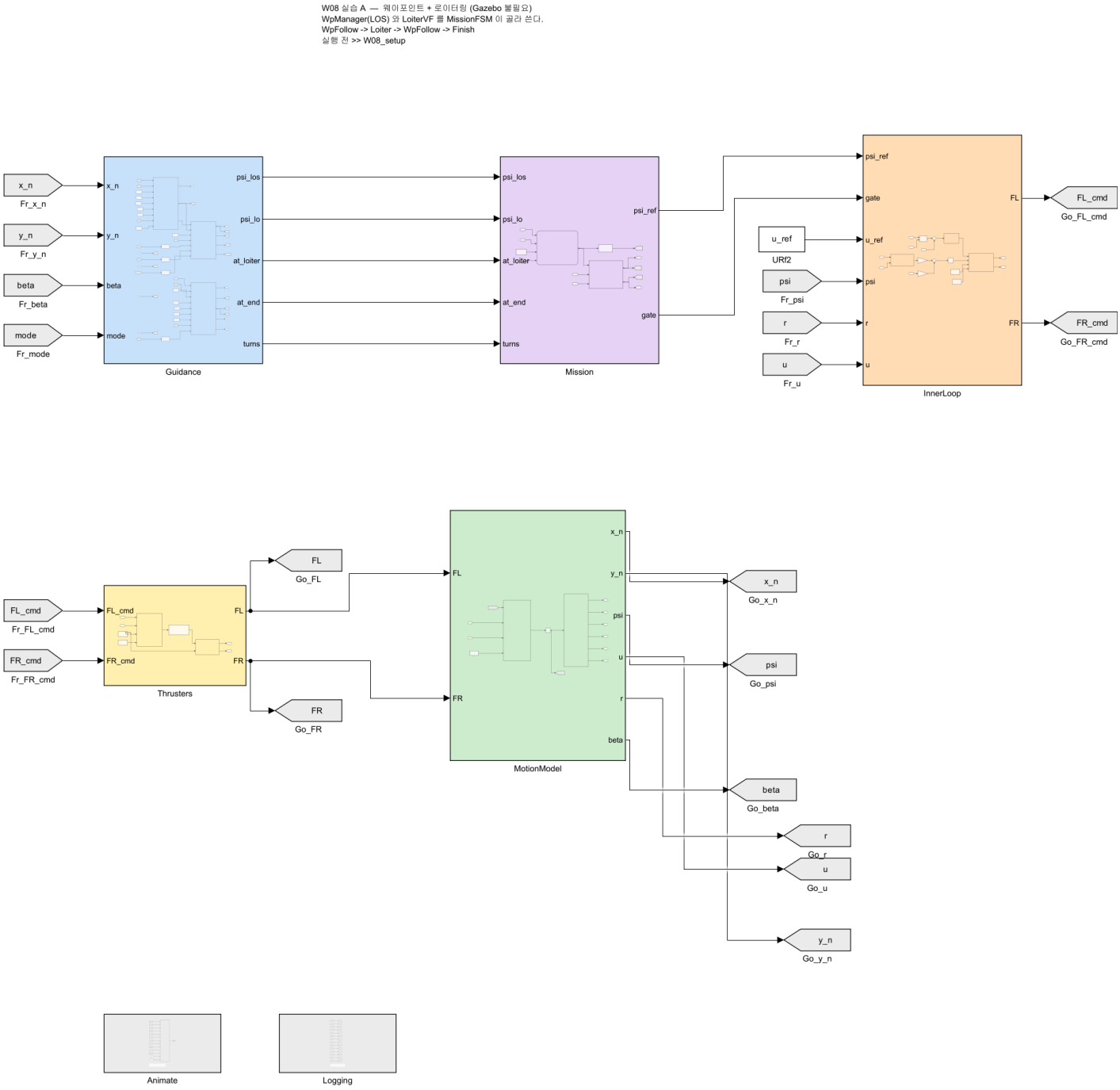
- 확인할 것
  - [ ] WP1 → WP2 로 올라감
  - [ ] WP2 → WP3 대각선으로 내려옴
  - [ ] **WP3 에서 배가 주황으로 바뀌고 원을 돎** — ① 의 계단이 `2 Loiter` 로 올라감
  - [ ] 2바퀴 후 파랑으로 돌아가 WP4 로 향함 — ① 의 계단이 `1 WpFollow` 로 내려옴
  - [ ] **로이터 원 밖에서 출발해 다시 경로로 붙음**
  - [ ] WP5 에서 회색으로 바뀌고 멈춤 — ① 의 계단이 `3 Finish` 로 올라가고 ③ 의 지령선이 0 이 됨

## 성능 그림




| 그림 | 볼 것 |
|---|---|
| 궤적 | 파랑(항주)과 주황(로이터)이 구분됨 |
| **미션 상태** | 1 → 2 → 1 → 3 계단 |
| 웨이포인트 진행 | 로이터 구간에서 **평평**한지 |
| 경로 오차 | 상태마다 다른 오차를 씀 |
| 회전수와 속도 | 로이터 구간에만 회전수가 오름 |

# B. 모델 구조 읽기 (20분)


W08 실습 A — 웨이포인트 + 로이터링 (Gazebo 불필요)
WpManager(LOS) 와 LoiterVF 를 MissionFSM 이 골라 쓴다.
WpFollow -> Loiter -> WpFollow -> Finish
실행 전 >> W08_setup
x_n
Fr_x_n
y_n
Fr_y_n
beta
Fr_beta
mode
Fr_mode
psi_los
psi_lo
at_loiter
at_end
turns
Guidance
psi_ref
gate
Mission
u_ref
URf2
psi
Fr_psi
r
Fr_r
u
Fr_u
FL
FR
InnerLoop
FL_cmd
Go_FL_cmd
FR_cmd
Go_FR_cmd
Fr_FL_cmd
Fr_FR_cmd
Thrusters
Go_FL
Go_FR
x_n
y_n
psi
u
r
beta
MotionModel
Go_x_n
Go_psi
Go_beta
Go_r
Go_u
Go_y_n
Animate
Logging

| 블록 | 출처 | 하는 일 |
|---|---|---|
| `WpManager` | 5주차 확장 | LOS + 인덱스 관리 + 도착 판정 |
| `LoiterVF` | 7주차 그대로 | 벡터필드 유도 |
| `TurnCount` | 7주차 수정 | 로이터 모드일 때만 회전수 누적 |
| **`MissionFSM`** | **신규** | Stateflow 차트 — 미션 상태기계 |
| **`ModeSwitch`** | **신규** | 두 유도 중 하나 선택 |
| `ModeDly` | 신규 | 대수 루프를 끊는 단위지연 |
| `InnerLoop` | 5주차 그대로 | 헤딩 P-D(연속) + 속도 PI(연속) + 배분 |
| `Thrusters` | 5주차 그대로 | 프로펠러·모터 (1차 지연 연속) |
| `MotionModel` | 5주차 그대로 | 운동방정식 + 적분 |

> ★ **제어기와 동특성은 전부 연속 시간임 (2026-10-01)**
>
> - 헤딩 D 항은 요각속도 되먹임 하나임. `Kd_rate` 는 **양수**이고 빼는 일은 `SumN` 이 `+-` 로 함 (4주차 2-8절)
> - 속도 PI 는 Continuous-time, 모터 지연은 연속 `State-Space` , 솔버는 두 모델 모두 `ode4`
> - 이산으로 남은 것은 **대수 루프를 끊는 단위지연 다섯 개**뿐임 — `IdxDly` (웨이포인트 번호), `ThDly` · `AccDly` · `ArmDly` (회전수 계수기), `ModeDly` (FSM 모드). 모두 정수·사건 상태라 적분할 것이 없음

> ★ **최상위에는 상자 다섯 개만 보인다**
>
> `Guidance` -> `Mission` -> `InnerLoop` -> `Thrusters` -> `MotionModel`
>
> - `WpManager` · `LoiterVF` · `TurnCount` 는 **`Guidance` 상자 안**에 있음
> - `MissionFSM` · `ModeSwitch` · `ModeDly` 는 **`Mission` 상자 안**에 있음
> - 웨이포인트·로이터 중심·요구 회전수 같은 **설정값도 각 상자 안**에 있음
> - 유도는 두 지령을 **둘 다** 내놓고, 고르는 일은 미션 상자만 함
>
> 그래서 유도(파랑)와 판단(보라)이 색으로도 갈림

## 블록 색으로 역할을 구분한다

> ℹ **5~8주차의 모든 모델이 같은 색 규칙을 따른다**

| 색 | 역할 | 예 |
|---|---|---|
| 파랑 | 유도 (Guidance) | `Guidance` , `WpManager` , `LoiterVF` , `TurnCount` |
| 보라 | 미션 판단 | `Mission` , `MissionFSM` , `ModeSwitch` |
| 주황 | 제어 (Control) | `InnerLoop` , PID, 추력배분 |
| 노랑 | 추진기 | `Thrusters` |
| 초록 | 운동모델 (Plant) | `MotionModel` |
| 연보라 | ROS 통신 | `PoseSubscriber` , `CmdPublisher` |
| 회색 | 로깅·표시 | `Logging` , `Animate` , Goto/From 태그 |
| 흰색 | 설정값 | Constant |

- 선은 **직선 또는 직각**만 씀. 대각선 없음

- 되먹임처럼 뒤로 돌아가는 선만 꺾임
- 배치를 흐트러뜨렸으면 생성 스크립트를 다시 돌림 (배치까지 다시 만듦)

> **✕ 생성 스크립트는 모델 파일을 지우고 새로 만든다**
>
> - `build_w08_models` 는 `W08_0_offline.slx` · `W08_1_vrx.slx` 를 **삭제한 뒤** 다시 만듦
> - 과제 ②·④ 에서 고친 내용이 **경고 없이 사라짐**
> - 모델을 고치기 전에 사본을 만들어 두고 (B-2 절 2단계), 과제 작업은 사본에서 할 것

```
W08_setup
build_w08_models
```

- 정상 출력 — 끝에 아래 세 줄이 찍히면 정상 (앞의 배치·색 점검 메시지는 생략)
  - 폴더에 다른 `W08_*.slx` 사본이 있으면 그 이름도 함께 찍힘

```
완료. 생성된 모델:
  W08_0_offline.slx
  W08_1_vrx.slx
```

## B-1. Stateflow 차트 열어 보기

```
open_system('W08_0_offline/Mission/MissionFSM')
```

- Stateflow 편집기 창이 열리고 상태 세 개가 보이면 정상
- 상태 세 개, 전이 네 개 (기본 전이 포함)
- 각 상태의 `en:` 는 **그 상태에 들어갈 때 한 번** 실행됨
- 전이 조건은 대괄호 `[ ]` 안에 씀
- `WpFollow` 에서 나가는 두 전이의 시작점 옆 번호( `1` · `2` )가 실행 순서 (1-2절)

## B-2. 차트 직접 만들기 (25분)

- 목표: 비어 있는 차트에 1-2절의 상태기계를 손으로 다시 그려, 원본과 같은 전이 시각을 얻음
- 원본은 건드리지 않고 **사본**에서 작업함

1. 설정을 불러옴 (모델이 한 번 돌고 그림이 뜸)

```
W08_setup
```

2. 오프라인 모델의 사본을 만듦 — 이후 모든 작업은 `W08_my` 에서

```
open_system('W08_0_offline')
save_system('W08_0_offline', 'W08_my')
```

3. 사본의 차트를 엶

```
open_system('W08_my/Mission/MissionFSM')
```

4. 차트 안의 상태·전이를 모두 지움 — 빈 곳 클릭 → `Ctrl` + `A` → `Delete`
   - 데이터(입력·출력)는 지워지지 않고 남음
5. **Modeling** 탭 → **Symbols Pane** 을 열어 데이터 다섯 개가 1-2절「차트의 데이터」표와 같은지 확인
   - 입력 `at_loiter` (1) · `turns` (2) · `req_turns` (3) · `at_end` (4), 출력 `mode` (1)
   - 빠진 것이 있으면 Symbols 창의 **Add Data** 로 추가하고 범위(Input/Output)와 포트 번호를 맞춤

6. 차트 속성 확인 — 빈 곳 우클릭 → **Properties**
   - Action Language `MATLAB`, Update method `Discrete`, Sample time `Ts_ctrl`
7. 왼쪽 도구 모음의 **State** 로 상태 세 개를 그리고, 상태 안에 두 줄씩 적음

| 위치 | 상태 안에 적을 글 |
|---|---|
| 왼쪽 위 | `WpFollow` ↵ `en: mode = 1;` |
| 왼쪽 아래 | `Loiter` ↵ `en: mode = 2;` |
| 오른쪽 위 | `Finish` ↵ `en: mode = 3;` |

8. 왼쪽 도구 모음의 **Default Transition** 으로 `WpFollow` 위쪽 가장자리를 클릭 → 기본 전이
9. 전이 세 개를 **아래 순서대로** 그림 — 상태 가장자리에서 다른 상태로 끌고, 화살표의 `?` 를 클릭해 조건을 적음

| 순서 | 출발 → 도착 | 조건 |
|---|---|---|
| ① | `WpFollow` 아래쪽 → `Loiter` 위쪽 | `[at_loiter > 0.5]` |
| ② | `Loiter` 오른쪽 → `WpFollow` 오른쪽 | `[turns >= req_turns]` |
| ③ | `WpFollow` 오른쪽 → `Finish` 왼쪽 | `[at_end > 0.5]` |

10. `WpFollow` 에서 나가는 두 전이의 번호 확인 — `→ Loiter` 가 `1`, `→ Finish` 가 `2`
    - 다르면 전이 우클릭 → **Execution Order** 에서 바꿈
11. 저장하고 실행

```
save_system('W08_my')
out = sim('W08_my');
S = W08_plot(out, 'W08_my');
```

12. 출력의 세 시각이 원본과 같은지 확인 — 로이터 진입 **74.6 s**, 종료 **247.9 s**, 임무 종료 **314.4 s**
    - 다르면 조건식의 오타, 데이터 포트 번호, 실행 순서를 차례로 확인

> ✎ **모델 이름만 바뀌었다**
>
> - `W08_my` 는 `W08_0_offline` 과 내용이 같고 이름만 다름
> - `W08_setup` 은 계속 `W08_0_offline` 을 돌림. 사본은 위처럼 `sim('W08_my')` 로 직접 돌릴 것

> ✎ **시뮬레이션 중 활성 상태 확인**
>
> Stateflow 차트를 열어 둔 채로 실행하면 활성 상태가 **파랗게 표시**됨
> 다만 그리는 데 시간이 걸려 느려짐. 실험할 때는 닫아 둘 것

---

## C. 미션 바꿔 보기 (30분)

### C-1. 설정 하나씩

| 바꿀 값 | 시도 | 예상 |
|---|---|---|
| `loiter_wp` | `2` / `4` | 다른 웨이포인트에서 돎. 반경 18 m 그대로면 **오프라인만** (VRX 에서는 `loiter_radius` 8 이하와 함께) |
| `loiter_radius` | `10` / `25` | 원 크기가 바뀜 |
| `loiter_turns` | `1` / `4` | 도는 횟수 |
| `p_c` | `-0.313` | 반시계방향으로 돎 |
| `crab_comp` | `0` | 로이터 반경 오차가 커짐 |
| `Delta` | `5` / `20` | 로이터 후 경로 복귀가 급해지거나 완만해짐 |

**✕ 로이터 원이 안전 수역을 벗어나면 안 된다**

- 안전 수역: $x \in [-28, 60]$ m, $y \in [-18, 100]$ m
- 원이 차지하는 범위: 중심 $(x_c, y_c)$, 반경 $r_d$ 이면 $x \in [x_c - r_d,\ x_c + r_d]$, $y \in [y_c - r_d,\ y_c + r_d]$

| `loiter_wp` | 중심 (x, y) | 반경 18 m 일 때 | 판정 |
|---|---|---|---|
| 2 | (50, 20) | $x$ 최대 68 | 벗어남 → 반경 8 이하 |
| **3 (기본)** | (20, 55) | $x \in [2, 38]$, $y \in [37, 73]$ | 안전 |
| 4 | (50, 90) | $x$ 최대 68, $y$ 최대 108 | 벗어남 → 반경 8 이하 |

### C-2. 로이터를 두 번 하려면

- 지금 상태기계는 로이터를 **한 번만** 함
- 두 번 하려면 상태를 늘리거나, `loiter_wp` 를 배열로 만들어야 함
- 과제 ④에서 직접 해 볼 것

---

## D. VRX 연동 (30분)

- 5·7주차와 절차가 같음

```
ros2 launch vrx_gz competition.launch.py \
    world:=sydney_regatta \
    urdf:=$HOME/capstone_ws/wamv/w7_wamv.urdf.xacro
```

```
setappdata(0, 'W08_skip_run', true);   % W08_setup 의 오프라인 자동 실행을 건너뜀
W08_setup
W08_vrx_run(420)
```

- `W08_vrx_run.m` 이 하는 일

| 단계 | 화면에 찍히는 줄 | 내용 |
|---|---|---|
| 0 | `0) ROS_DOMAIN_ID = 8` | Simulink 프로필과 MATLAB 의 도메인을 맞춤 |
| 1 | `1) VRX 토픽 확인` | ground truth 토픽이 없으면 여기서 멈춤 |
| 2 | `2) RTF 측정 (20초)` · `스폰 위치 (x, y) = ...` | 실측 RTF 로 페이싱을 정하고, **배가 지금 있는 자리를 원점으로** 삼음 |
| 3 | `3) W08_1_vrx 실행 (420초)` | 실시간 화면 네 칸 (D-3). 상태가 바뀌면 배 색이 바뀌고 ① 의 계단이 올라감 |
| 4 | `4) 결과 그림` · `===== VRX 미션 ... =====` | VRX 지표 5줄, 그림 `img/W08_1_vrx_result.png` |
| 5 | `5) 같은 초기 선수각으로 오프라인 모델 실행` | 오프라인 지표 5줄, 대조 그림 `img/W08_vrx_vs_offline.png` |

**⚠ 인자 없이 부르면 300초 에서 끊긴다**

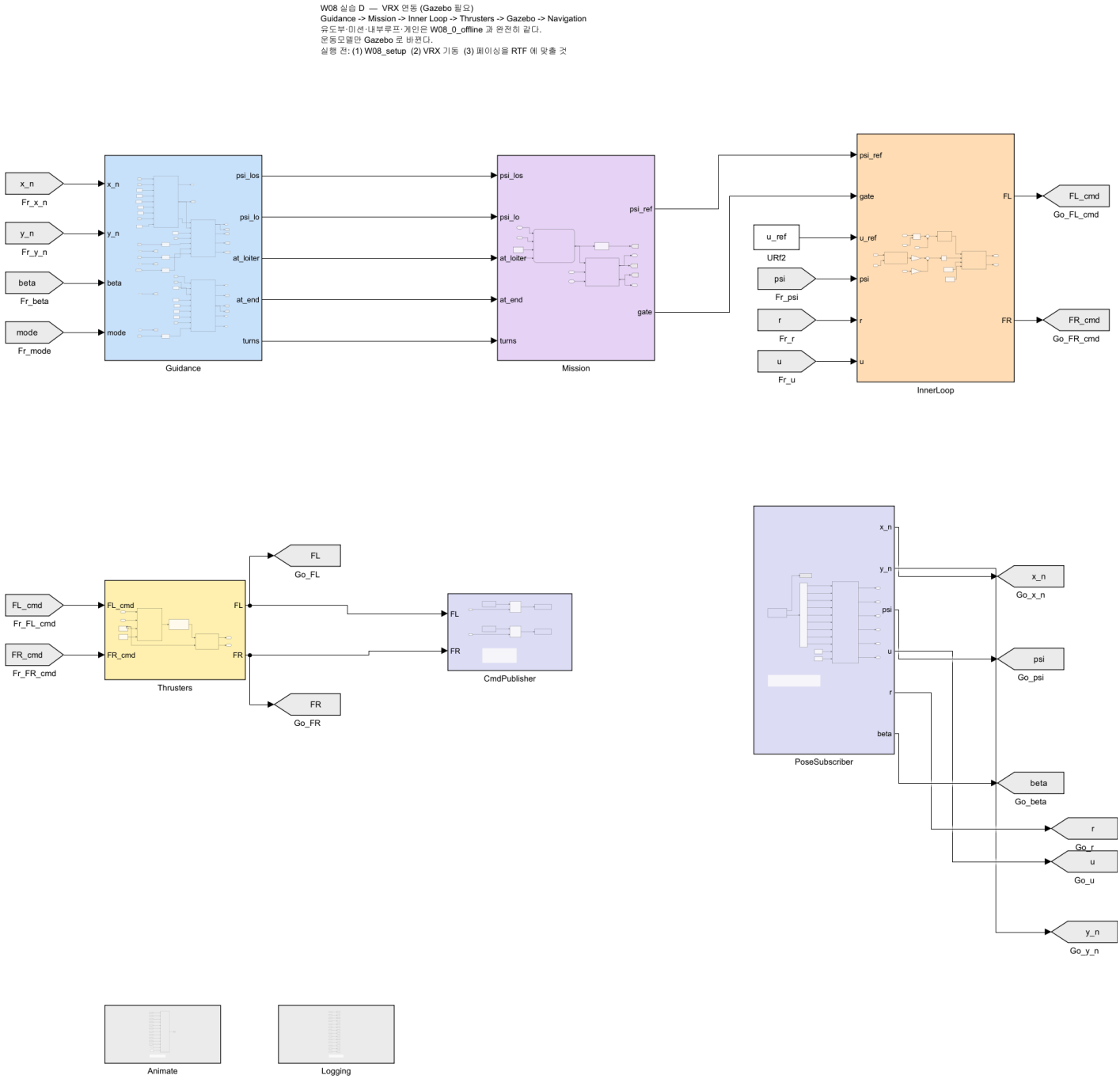
- 임무 종료가 약 300~314 s 라 300 s 로는 마지막 웨이포인트에 닿기 전에 끝남 → `임무 종료 미완료`
- `W08_vrx_run(420)` 처럼 여유를 두고 부름
- 전이 시각은 **시뮬레이션 시각**임. 페이싱이 실측 RTF 와 맞으면 느린 컴퓨터에서도 같은 시각이 나옴. 벽시계로는 더 오래 걸릴 뿐

**★ VRX 를 새로 띄운 뒤 시작한다**

- 앞 실험이 끝난 자리에서 이어 돌리면 첫 웨이포인트까지의 거리가 달라져 시각 비교가 무의미해짐
- RTF 가 1 % 미만인 노트북은 `"extra_gz_args:=--render-engine-server ogre --render-engine-gui ogre"` 를 함께 줌

**✕ 스폰 위치는 설치마다 다르다 — 스크립트가 직접 잰다**

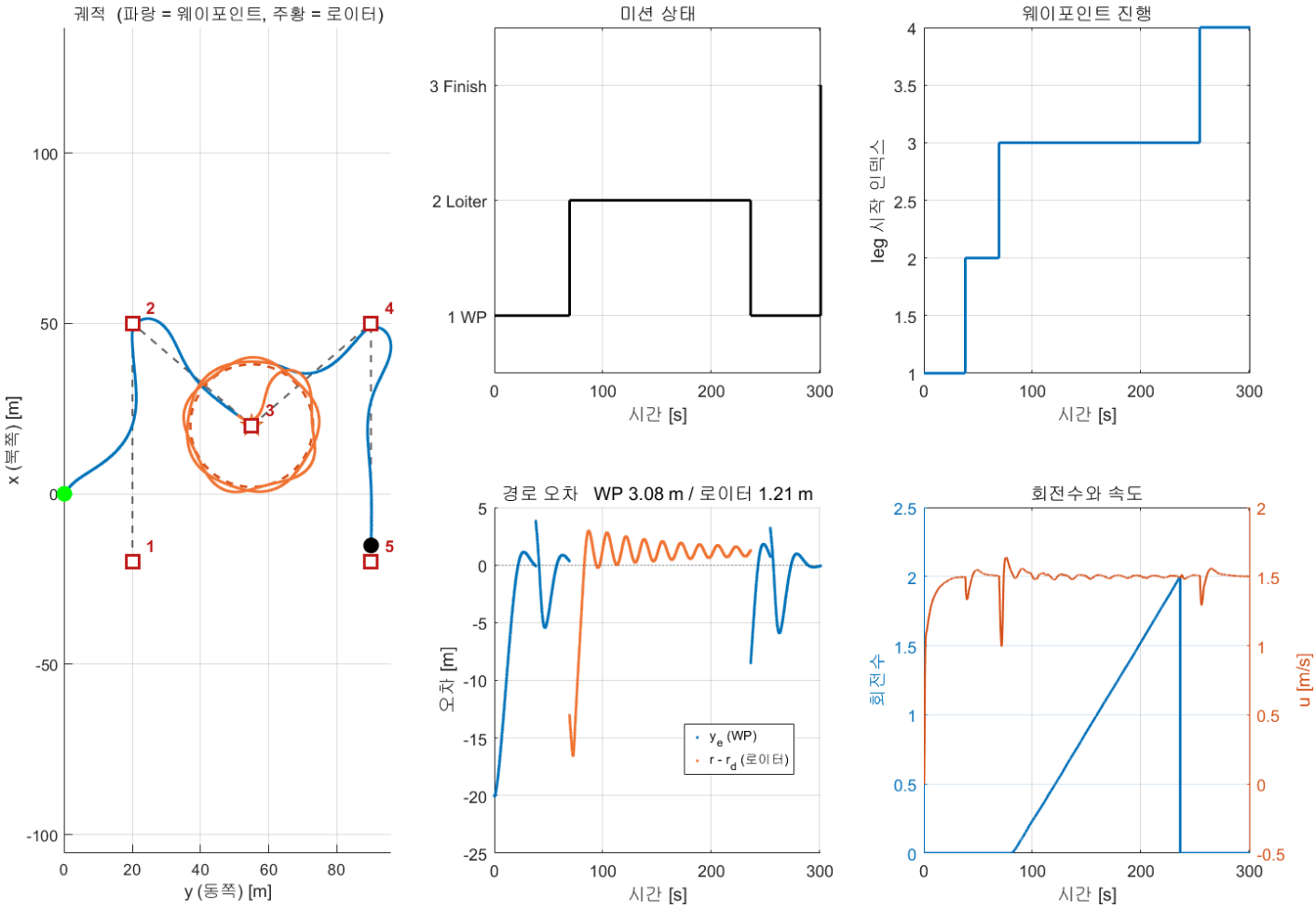
- `competition.launch.py` 의 스폰 좌표가 같은 VRX 2.4.0-2 인데도 컴퓨터마다 달랐음: $y_{\mathrm{ENU}}$ = **162** 인 설치와 **200** 인 설치
- `W08_setup.m` 의 `origin_north = 162` 는 기본값일 뿐. `W08_vrx_run` 이 ground truth 첫 샘플로 **덮어씀** (2단계 출력의 `스폰 위치` )
- 고정값 162 를 200 인 설치에서 그대로 쓰면 경로 전체가 **38 m 남쪽으로** 밀림. 실측 결과
  - 출발점이 $(x, y) = (38, 0)$ 으로 찍히고, WP4 → WP5 구간이 안전 수역 $x \geq -28$ 밖으로 나가 육지에 걸려 **멈춤** ($x \approx 1$ 에서 정지, 추력 한계)
  - 궤적 평균 이격 25.70 m — 경로 오차가 아니라 원점 오차




## D-1. 결과

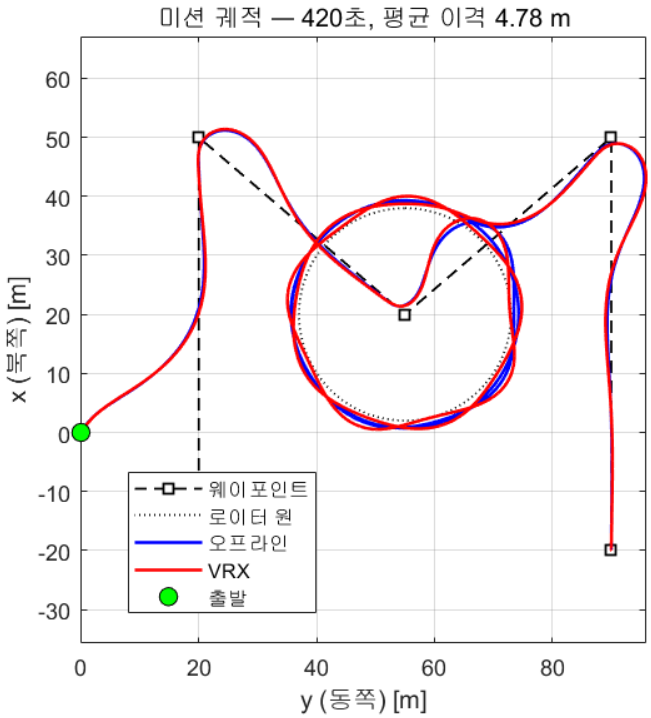
- 기준 환경 실측값 (2026-09-19, 데스크톱, 측정 RTF 0.668, 420 s, 스폰 $(x_{\mathrm{ENU}}, y_{\mathrm{ENU}}) = (-532, 200)$)

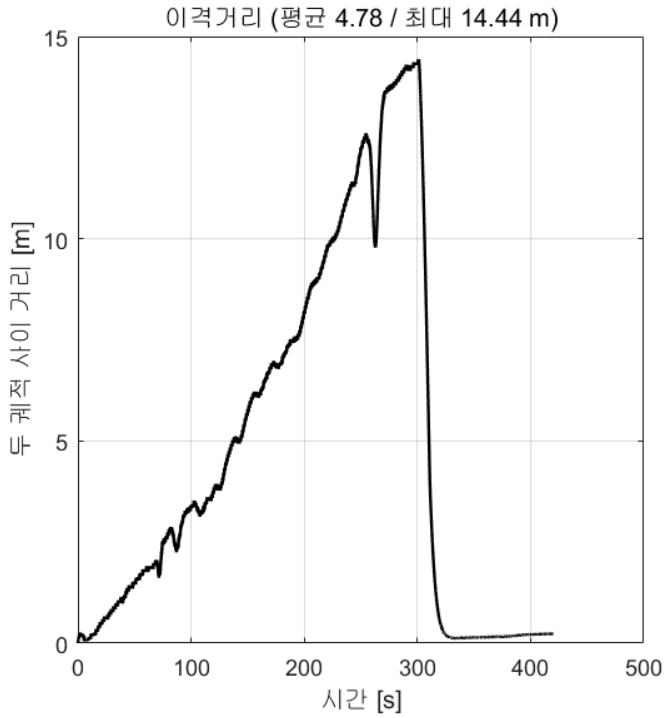
```
===== VRX 미션 — WP -> 로이터 -> WP =====
  로이터 진입              69.4 s
  로이터 종료             236.4 s   (2.00 바퀴)
  임무 종료               301.1 s
  WP 구간 평균 |y_e|       3.077 m
  로이터 평균 반경오차     1.213 m
```




| 읽는 법 | |
|---|---|
| 상태 1 → 2 → 1 → 3 | 오프라인과 같은 순서. 궤적 그림의 파랑·주황이 같은 자리에서 바뀜 |
| 회전수 2.00 바퀴 | 종료 조건이 정확히 동작 |
| 로이터 반경오차 1.213 m | 오프라인 1.282 m 와 0.07 m 차이. **상태기계와 유도는 시뮬레이터가 바뀌어도 같게 동작** |
| WP 구간 $\lvert y_e \rvert$ 3.077 m | 오프라인 2.957 m 보다 0.12 m 큼. 대부분이 로이터 직후 복귀 구간 |

## D-2. 오프라인과 비교




- 오프라인 열은 `W08_vrx_run` 이 **VRX 의 초기 선수각(32.69°)으로 다시 돌린** 값임
  - 그래서 `W08_setup` 기본 실행(초기 선수각 135°, 진입 74.6 s)과 다름
  - `W08_vrx_run` 은 끝나면 `x0` 를 원래대로 되돌림. 뒤이어 오프라인을 돌려도 기본 조건 그대로

| 항목 | 오프라인 | VRX | 차이 |
|---|---|---|---|
| 로이터 진입 [s] | 70.8 | 69.4 | −1.4 |
| 로이터 종료 [s] | 244.1 | 236.4 | −7.7 |
| 회전수 | 2.00 | 2.00 | 0 |
| 임무 종료 [s] | 310.6 | 301.1 | −9.5 |
| 로이터 평균 반경오차 [m] | 1.282 | 1.213 | −0.069 |
| 궤적 평균 이격 [m] | | | 4.78 |
| 궤적 최대 이격 [m] | | | 14.44 |

- 상태 순서 · 회전수 · 반경오차는 같음. 미션 모델은 이 셋으로 판정함
- 이번 실행은 VRX 가 오프라인보다 **9.5 s 빨리** 끝남 → 이격 4.78 m 는 **위상 오차**
  - 궤적 그림에서 두 선은 겹침. 같은 길을 다른 시각에 지나므로 같은 시각끼리 빼면 멀어짐 → 궤적이-겹치면-이격은-오차가-아니라-지연이다
  - 같은 설정의 2026-09-18 실행은 전이 시각이 1 s 안, 임무 종료 +2.1 s, 이격 0.83 m 였음. 반경오차는 1.211 m 로 이번과 같음
- 이격과 전이 시각 차이는 페이싱 비율(측정 RTF)과 실제 진행이 얼마나 어긋났느냐에 따라 실행마다 달라짐

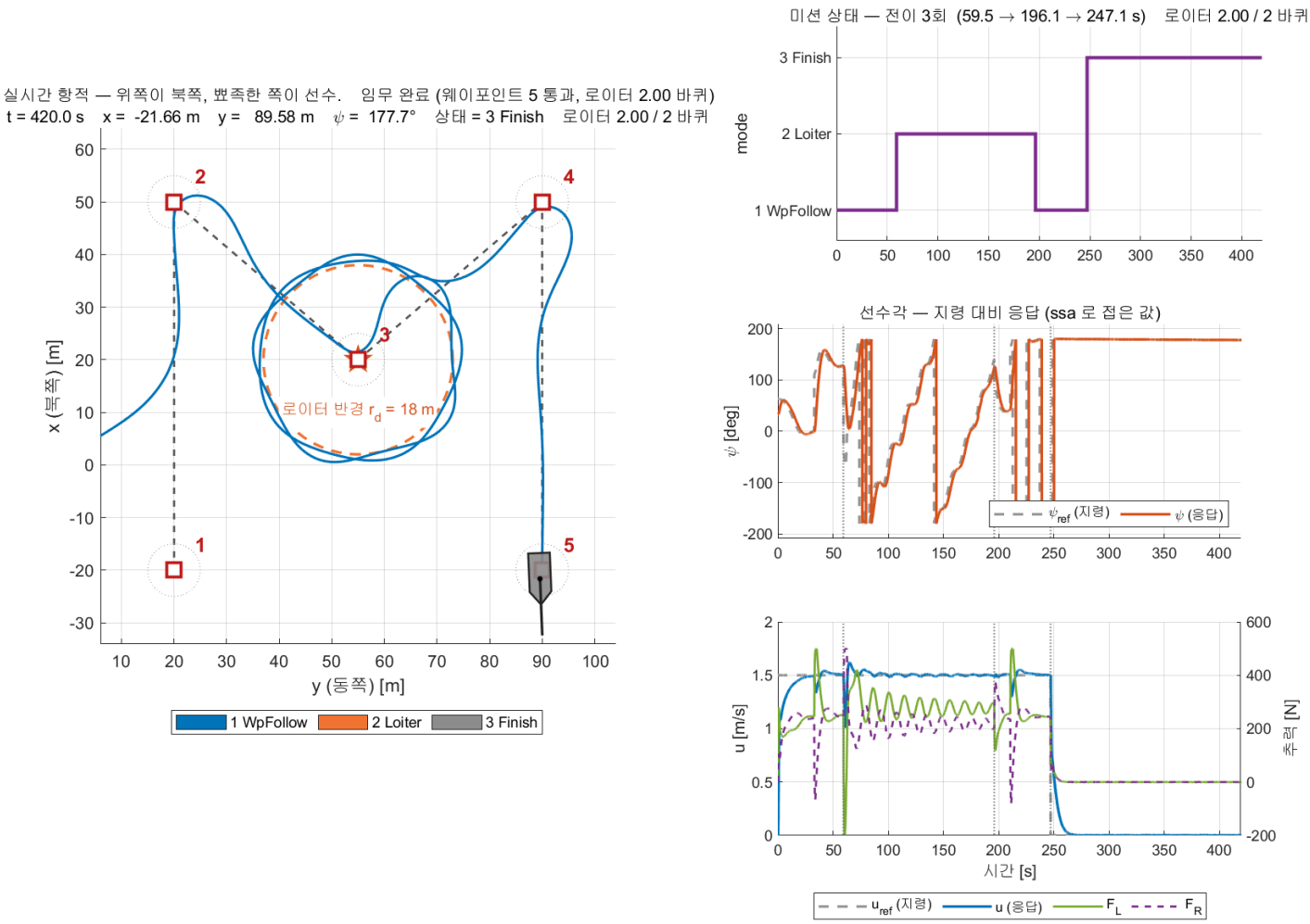
### ℹ 전이 시각이 실행마다 흔들리는 이유

- 오프라인은 VRX 초기 선수각으로 다시 돌리므로 **출발 조건 차이는 아님**
- 가장 큰 몫은 페이싱 — 20초 동안 잰 RTF 가 주행 내내 유지되지 않으면 VRX 쪽 시각이 오프라인보다 앞서거나 뒤짐 (09-18 은 +2.1 s, 09-19 는 −9.5 s)
- 나머지는 두 모델의 차이 — VRX 는 odometry 가 약 10 Hz 로 들어오고(제어는 20 Hz), 부력·파랑 없는 수면이라도 선체 롤·피치가 있음
- 이런 편차는 실제 배에서도 생김. 상태기계가 **시각이 아니라 조건으로** 전이하도록 짜야 하는 이유

## D-3. 실시간 화면 — VRX

### ✎ 주행 중 실시간 화면이 표시됨

VRX 모델에도 오프라인과 같은 `Animate` 상자가 들어 있고 **같은 함수**( `W08_animate.m` )가 그림
배가 어디쯤 가고 있는지, 지금 어느 상태인지 Gazebo 화면 없이도 보임. 끄려면 `animate = 0`




- 조건 — `W08_1_vrx` , `W03_vrx_lite/run_vrx.sh lite` , `ROS_DOMAIN_ID = 7` ,
  측정 RTF 1.070 (페이싱도 같은 값), 스폰 $(x_{\mathrm{ENU}}, y_{\mathrm{ENU}}) = (-532.1, 200.0)$,
  420 s 주행 / 벽시계 546.8 s, 게인·미션 설정은 A절과 같음 (2026-09-25 실측)
- 캡처 명령 — `W08_anim_capture('vrx')`

| 항목 | 이 실행 (09-25) | D-1 기준 실행 (09-19) |
|---|---|---|
| 측정 RTF | 1.070 | 0.668 |
| 상태 전이 순서 | 1 → 2 → 1 → 3 | 1 → 2 → 1 → 3 |
| 로이터 진입 [s] | 59.5 | 69.4 |
| 로이터 종료 [s] | 196.1 | 236.4 |
| 회전수 | 2.00 | 2.00 |
| 임무 종료 [s] | 247.1 | 301.1 |

- **순서와 회전수는 같고 시각만 다름** — 위「전이 시각이 실행마다 흔들리는 이유」가 그대로 보이는 예임
  - 두 실행의 차이는 페이싱 비율임 (0.668 대 1.070). 이 컴퓨터는 이번에 RTF 가 1 을 넘었음
  - 그래서 임무 종료가 D-1 보다 54 s 이름. D-1 의 지표 수치는 **그 실행의 것으로 그대로 둠**
- 네 칸의 뜻과 읽는 법은 A절「실시간 창 — 네 칸의 뜻」과 **완전히 같음.** 나란히 놓고 읽으면 됨
  - ① 의 계단이 같은 순서로 세 번 바뀌는지
  - ① 제목의 바퀴 수가 로이터 구간에서 2.00 까지 오르는지
  - 복귀 후 왼쪽 칸의 초록 원이 WP4 를 다시 집는지
  - ③ 에서 종료 뒤 추력이 0 으로 내려앉고 $u$ 가 따라 내려오는지

> **VRX 화면의 ③ 은 오프라인보다 추력이 잘게 흔들림**
>
> - 로이터 구간에서 $F_L \cdot F_R$ 이 톱니처럼 진동함 — odometry 가 약 10 Hz 로 들어오는데 제어는 20 Hz 라 같은 측정값을 두 번 쓰는 스텝이 생기기 때문임
> - 상태 전이와 회전수는 이 진동에 영향받지 않음. 미션 모델은 그 둘로 판정함 (D-2)

### 확인할 것

- [ ] 2단계 출력의 `스폰 위치` 가 launch 파일의 스폰 좌표와 같은가
- [ ] 상태 전이가 오프라인과 같은 순서로 일어나는가 — 실시간 창 ① 의 계단으로 바로 읽힘
- [ ] 전이 시각이 비슷한가 (페이싱에 따라 수 s 차이는 정상. 순서가 같아야 함)
- [ ] 로이터 반경 오차가 비슷한가
- [ ] 마지막에 두 궤적이 다시 만나는가

---

## E. 미션 끝에 접안 붙이기 (40분)

> **★ 실습 A ~ D 와 다른 모델·다른 설정 파일을 쓴다**
>
> - 모델 `W08_2_dock.slx` (오프라인) · `W08_3_dock_vrx.slx` (VRX)
> - 설정 `W08_dock_setup.m`
> - 실습 A ~ D 의 파일은 하나도 건드리지 않음. 둘을 번갈아 돌려도 서로 영향이 없음

### E-1. 오프라인 실행

```
cd('<배포 폴더>/W08_simulink')
W08_dock_setup
```

- 정상 출력

```
W08 실습 E 설정 완료
  웨이포인트 5개
  WP3 도착 시 로이터링 — 반경 18 m, 2 바퀴, 시계방향
  접안 위치 (x,y) = (-20, 75),  자세 180°,  접근점 (-20.0, 90.0)
  목표 속도 1.5 m/s, 접안 옆걸음 한계 0.6 m/s
  ROS_DOMAIN_ID = 7  (WSL 의 echo $ROS_DOMAIN_ID 와 같아야 함)

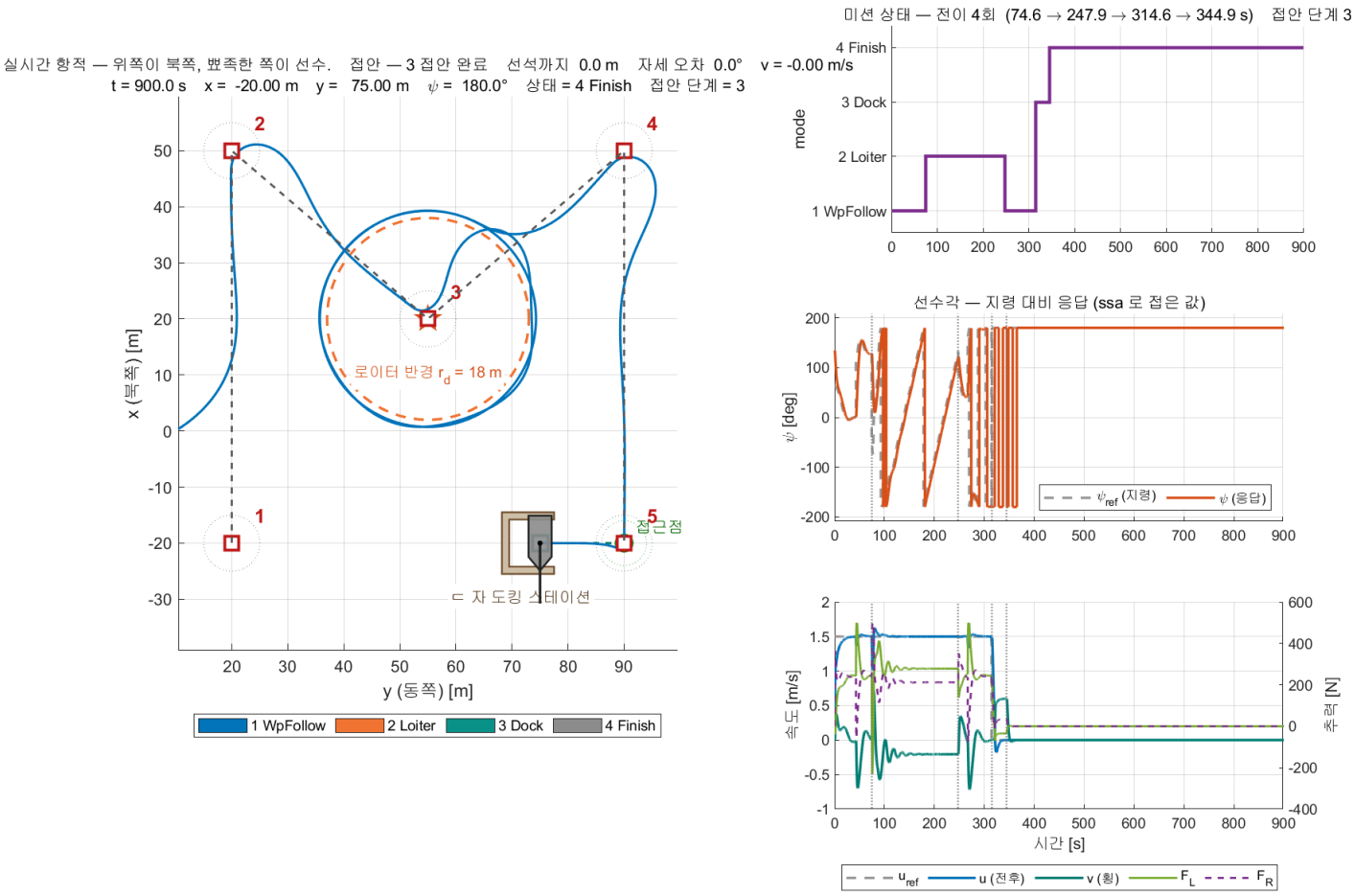
오프라인 모델 실행 중  (auto_run = 0 으로 두면 실행하지 않는다)

===== WP -> 로이터(WP3, 2바퀴) -> WP -> 접안 =====
  로이터 진입              74.6 s
  로이터 종료             247.8 s   (2.00 바퀴)
  접안 시작               314.6 s
  접안 완료               344.9 s   (접안에 30.4 s)
  WP 구간 평균 |y_e|      2.696 m
  로이터 평균 반경오차    1.282 m
  선석 위치 오차          1.467 m   (완료 문턱 1.50 m 을 넘은 순간)
  멈춘 뒤 위치 오차       0.079 m   (완료 +10 s, 그 사이 1.55 m 미끄러짐)
  옆걸음 선수각 오차      0.255 deg  (최대, phase 2 전 구간)
  옆걸음 |u| / |v|        0.192 / 0.599 m/s  (최대, phase 2 전 구간)
  선체 네 점              멈춘 뒤 모두 ㄷ 안
```

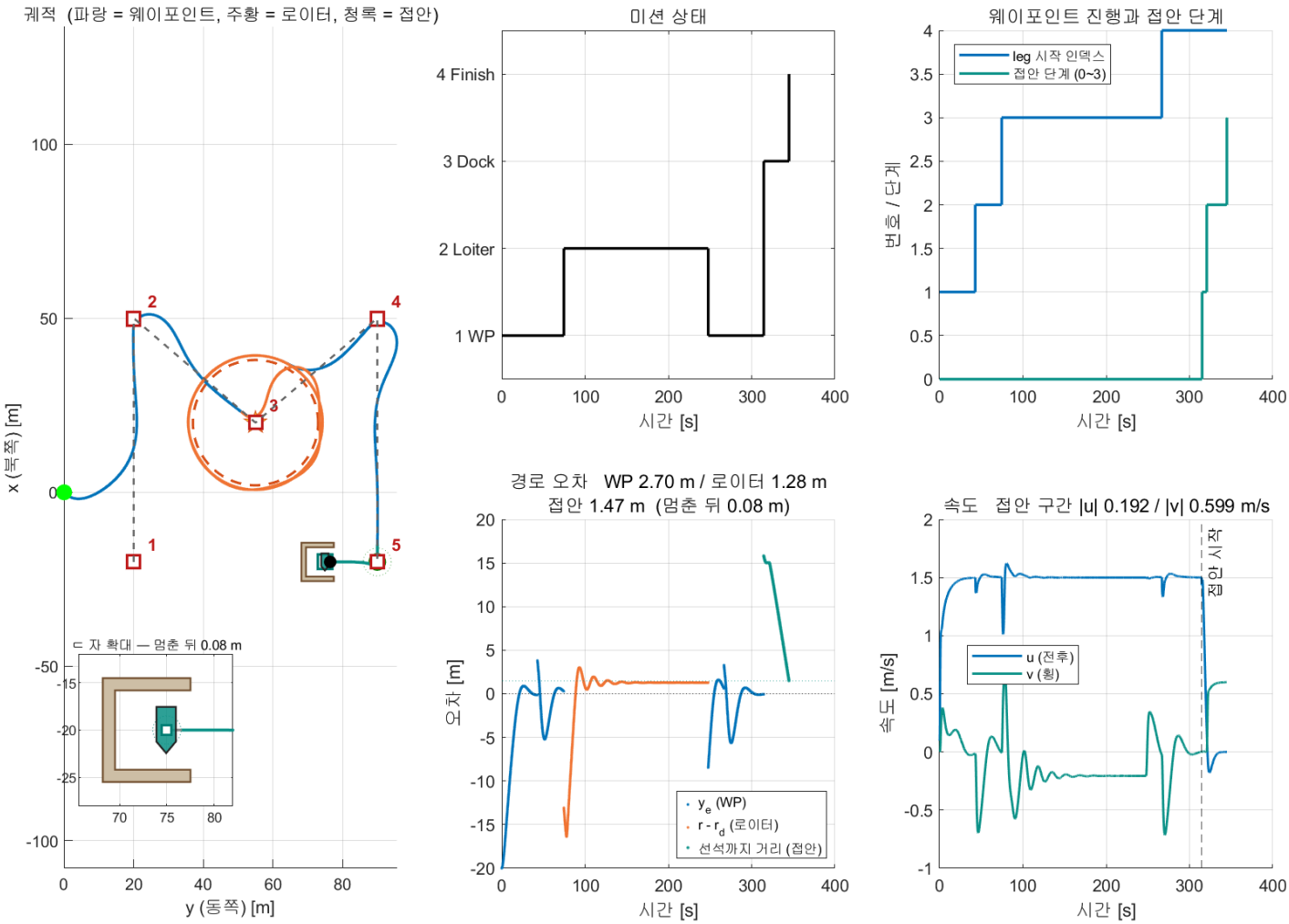
| 읽는 법 | 뜻 |
|---|---|
| 로이터 진입·종료 | 실습 A 와 같은 값임. 앞의 세 상태를 건드리지 않았다는 증거임 |
| 접안 시작 314.6 s | 실습 A 의 "임무 종료 314.4 s" 자리임. 같은 조건에서 `Finish` 대신 `Dock` 으로 들어감 |
| 접안에 30.4 s | 접근점 도달 → 자세 맞추기 → 15 m 옆걸음에 걸린 시간 |
| 선석 위치 오차 1.467 m | 완료 **판정 순간**의 값임. 문턱 1.50 m 을 막 넘은 값이므로 항상 1.5 m 근처임 |
| **멈춘 뒤 0.079 m** | 완료 +10 s 의 값임. **이 값이 접안 정밀도임.** 그 사이 1.55 m 더 미끄러져 들어가 제자리에 섬 |
| 선수각 오차 0.255° | 옆걸음 전 구간 최대값임. 1° 를 넘으면 순수 옆걸음이 아님 |
| $\lvert u\rvert$ 0.192 / $\lvert v\rvert$ 0.599 | 전후는 거의 0, 횡은 포화값 0.6 m/s — 옆으로만 갔다는 뜻임 |
| 선체 네 점 | 멈춘 자세의 선체 다각형 네 꼭짓점이 모두 베이 다각형 안에 드는지 `inpolygon` 으로 판정함 |

> ℹ **"선석 위치 오차" 와 "멈춘 뒤 위치 오차" 는 다른 것을 잰다**
>
> - 앞의 것은 **문턱을 넘은 순간** — 설정한 `R_stop` 이 1.5 m 이므로 늘 1.5 m 근처로 나옴. 성능 지표가 아님
> - 뒤의 것은 **완료 +10 s** — 위치제어가 자리를 잡은 뒤의 값임. 이것으로 평가함

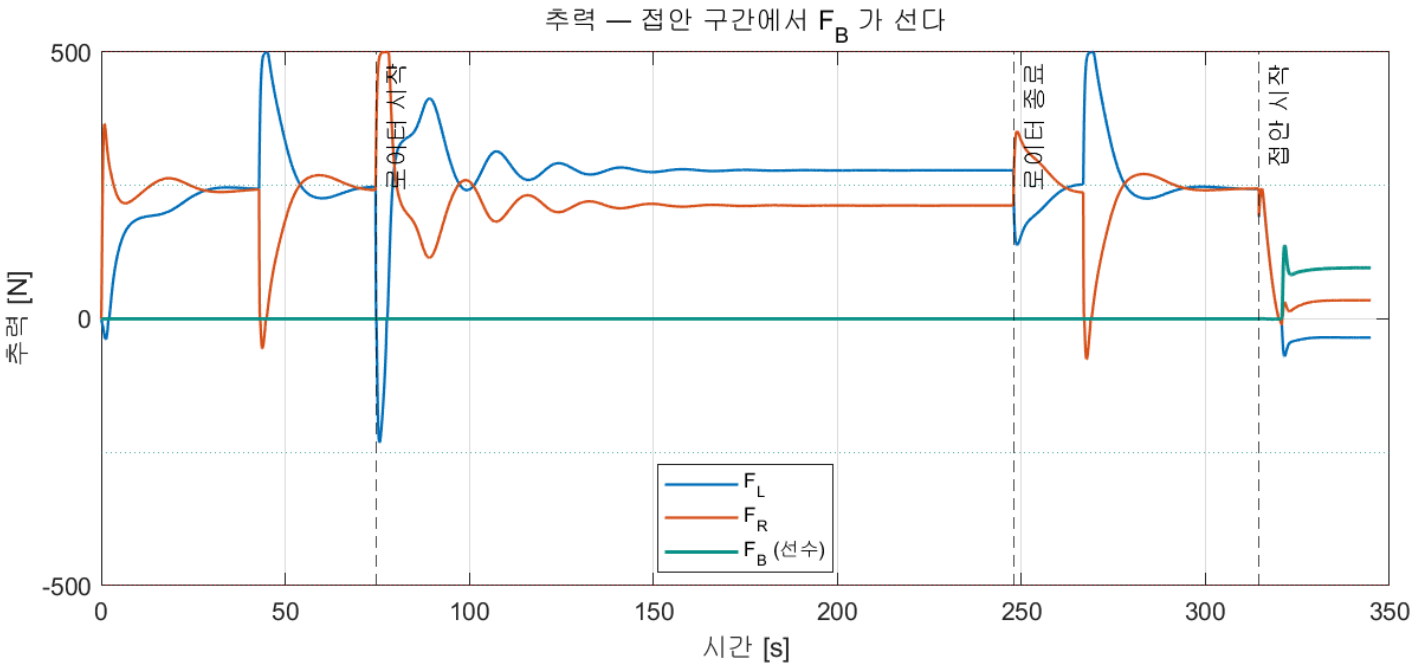
## 그림 세 장




- 실시간 화면은 실습 A 와 **같은 네 칸 배치**임. 접안 단계가 더해졌을 뿐임
  - 왼쪽 — 항적. 상태별 색이 **넷**으로 늘고, ㄷ 자 도킹 스테이션과 접근점이 그려짐
  - ① 미션 상태 — 계단이 **네 번** 바뀜. 제목에 전이 시각 네 개와 현재 접안 단계가 찍힘
  - ② 선수각 — 접안 뒤 180° 에 붙어 평평해짐
  - ③ 속도·추력 — 접안 구간에서 $u$ 가 0 으로 내려오고 $v$ 가 −0.6 m/s 에 섬

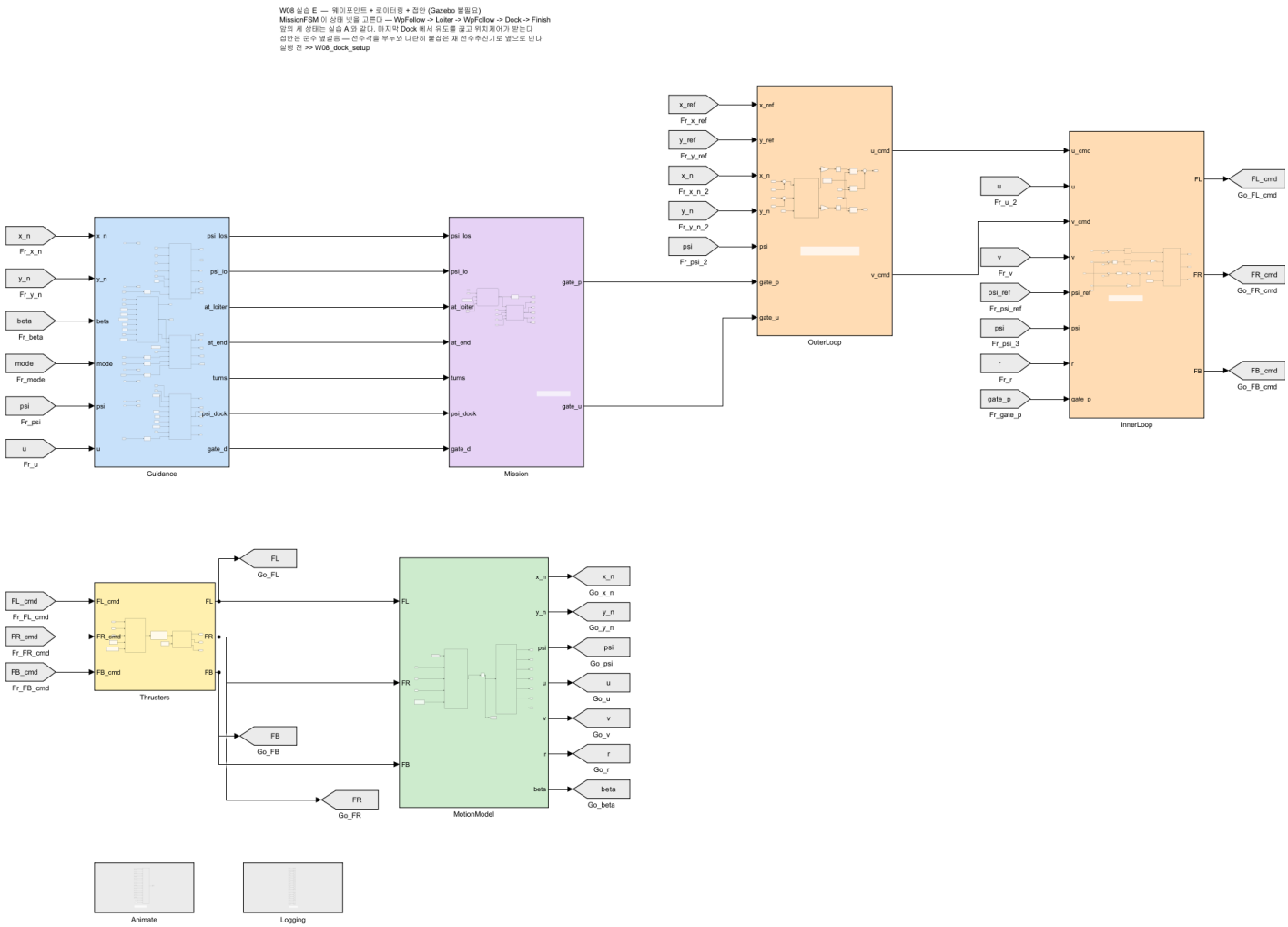



- 왼쪽 큰 칸 아래의 **확대 그림**이 ㄷ 자와 멈춘 선체를 함께 보여 줌. 전체 궤적 칸에서 ㄷ 은 10 m 짜리라 점으로 보임
- 가운데 아래「경로 오차」칸에 **선석까지 거리**(청록)가 더해짐. 15 m 에서 0 으로 내려옴
- 오른쪽 아래「속도」칸의 점선이 접안 시작 시각임




- 접안 구간에서 $F_B$**(선수추진기)가 서는 것**이 보임. 그 전까지는 0 임
- $F_L \cdot F_R$ 은 요 모멘트 보정만 하므로 크기가 작고 부호가 반대임

## E-2. 모델 구조 읽기 (15분)




- 실습 A 의 `W08_0_offline` 과 비교하며 읽음. 늘어난 것은 아래 넷뿐임

| 늘어난 것 | 어디에 | 하는 일 |
|---|---|---|
| `BerthRef` | `Guidance` | 접안 3단계 기준점 · 부두 선수각 · 위치제어 문 `gate_d` · 단계 `phase` |
| `PhaseDly` | `Guidance` | `phase` 를 자기에게 한 스텝 뒤로 되돌림 (걸쇠) |
| `OuterLoop` · `GateSway` | `Control` | 위치 오차 → 전후·횡 속도 지령 (6주차) |
| 스웨이 PI · 선수추진기 채널 | `Control` · `Thrusters` | 횡력 $Y$ 와 $F_B$ (6주차) |

- `Mission/MissionFSM` 차트를 열어 상태 넷과 전이 넷을 확인함
  - `Dock` 의 진입동작이 `mode = 3;` , `Finish` 가 `mode = 4;` 임
  - `Dock → Finish` 조건이 `[phase > 2.5]` 임

> ✎ **`To Workspace` 상자가 스무 개인 이유**
>
> - 실습 A 는 열한 개였음. 접안을 읽으려면 `phase` · `v` · `x_ref` · `y_ref` · `F_B` 가 더 있어야 함
> - 그림과 지표를 **실행 결과에서만** 만들기 위함임. 다시 계산하지 않음

## E-3. 설정 바꿔 보기 (10분)

| 바꿀 값 | 기본 | 해 볼 것 | 보이는 것 |
|---|---|---|---|
| `berth_side` | −1 (좌현) | +1 | 접근점이 반대쪽으로 가고 옆걸음 방향이 뒤집힘 |
| `dock_deg` | 180 | 90 | ㄷ 자가 돌아감. 안전 수역을 벗어나지 않는지 확인할 것 |
| `L_app` | 15 | 8 | 옆걸음 거리가 줄어 접안 시간이 짧아짐. 대신 자세 맞출 여유가 줄어듦 |
| `v_max` | 0.6 | 0.3 | 옆걸음이 느려짐. 접안 시간이 약 두 배가 됨 |
| `bow_on` | 1 | 0 | **선수추진기를 끔.** 횡력을 낼 수 없어 접안이 되지 않음 — 1-8 절의 이유를 눈으로 확인 |

- **끝나면 값을 원래대로 되돌려 놓을 것**

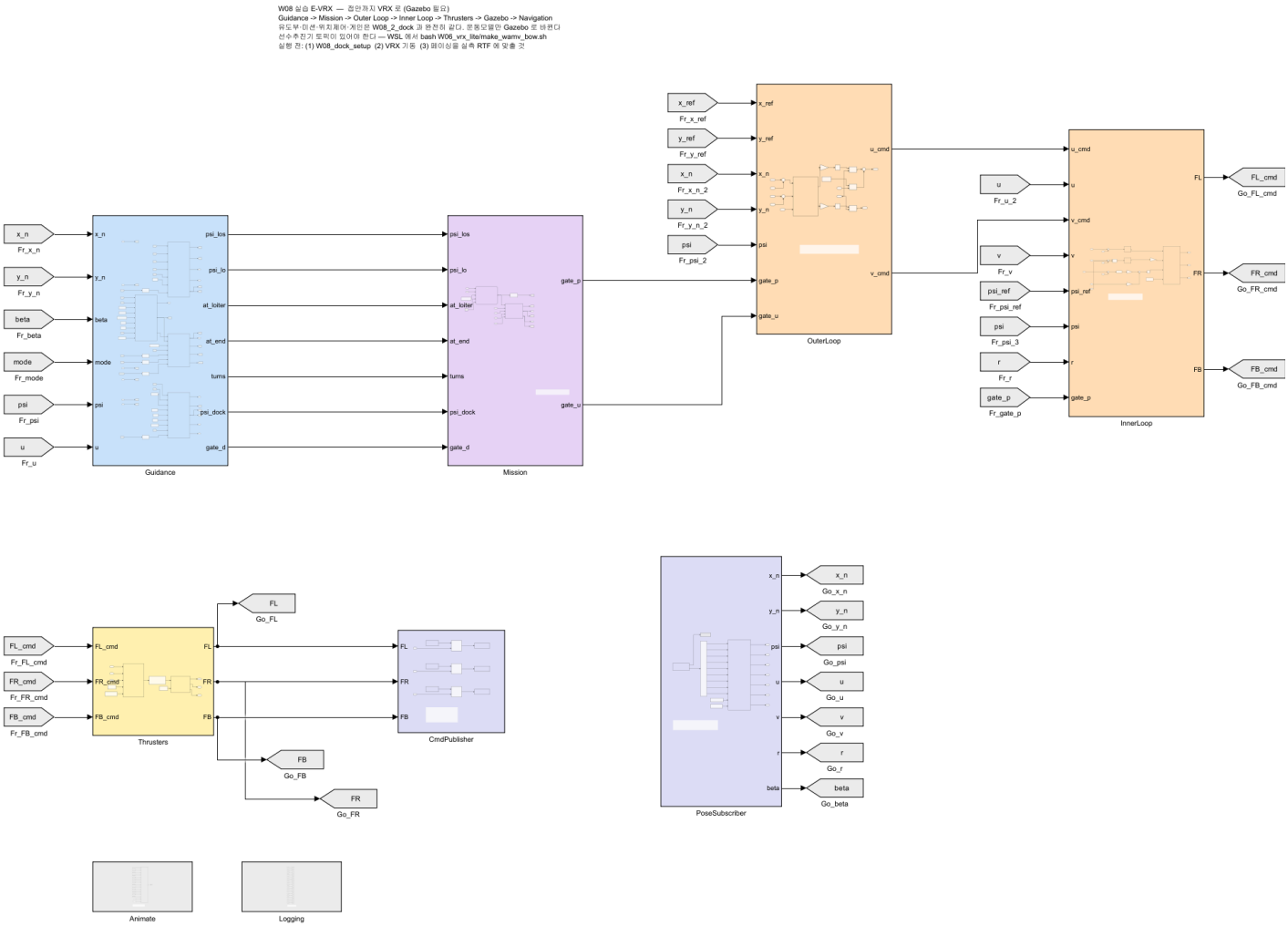
## E-4. VRX 검증 (15분)

> **★ 선수추진기 달린 WAM-V 를 띄운다**
> 기본 WAM-V 는 좌·우 추진기 둘뿐임. 실습 E 는 6주차에서 쓴 세 추진기 WAM-V 를 씀

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W06_vrx_lite
bash run_vrx_bow.sh lite
```

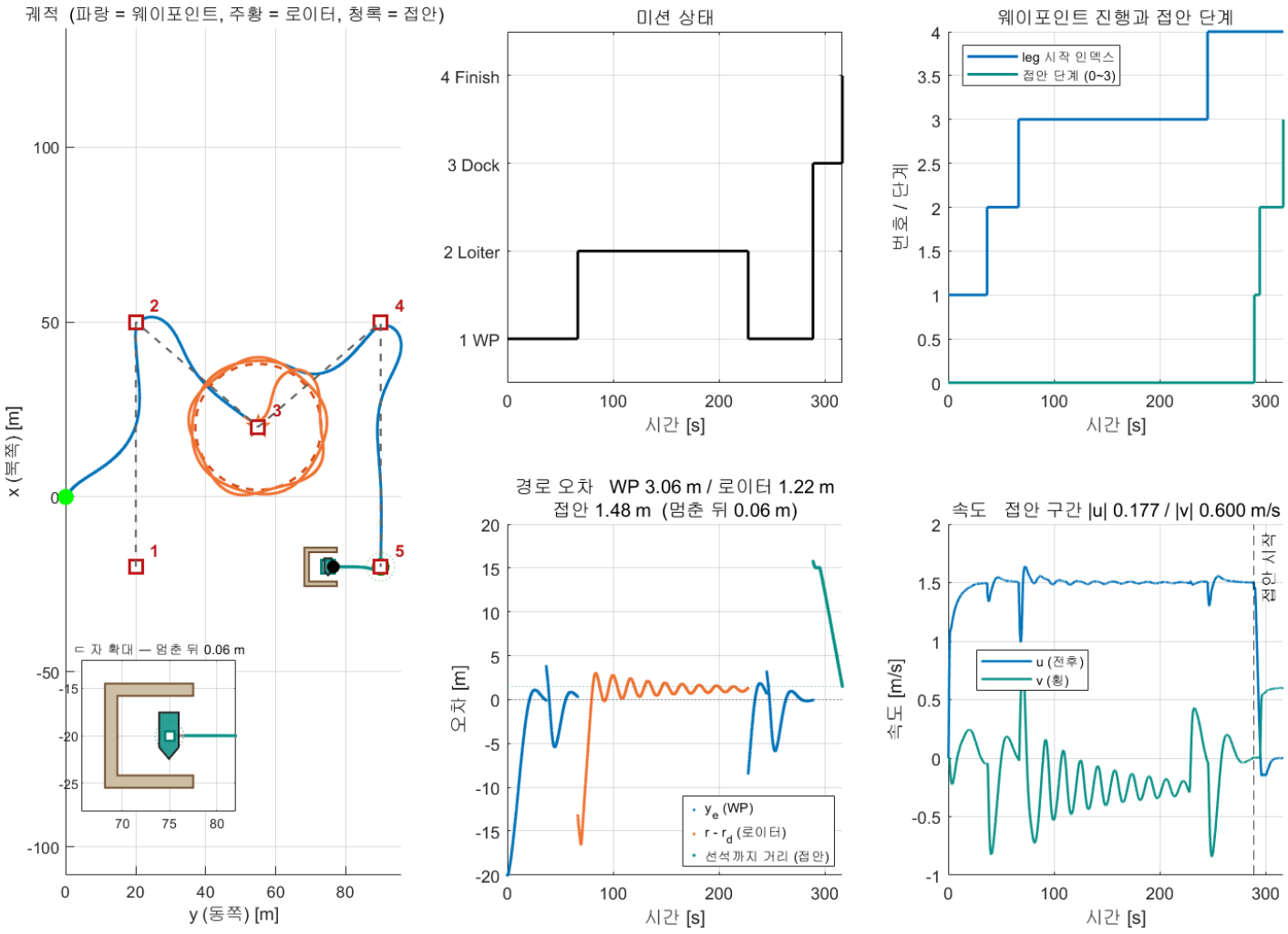
```
cd('<배포 폴더>/W08_simulink')
S = W08_dock_vrx_run(420)
```

> **i `W08_dock_setup` 을 먼저 돌릴 필요가 없다**
> - 설정이 안 깔려 있으면 `W08_dock_vrx_run` 이 스스로 깔고, 그때는 **오프라인 자동 실행을 건너뜀**
> - E-1 을 이미 돌렸다면 그 설정을 그대로 씀




- 오프라인 모델과 **제어기가 글자 하나까지 같음.** 사실 끝의 운동모델 자리만 ROS 통신으로 바뀜
  - `Nav` — ENU → NED 변환, 스폰 원점 빨셈, **첫 메시지 전 0 버스 가드**
  - `Thrust` — 좌·우·**선수** 추진기 세 토픽 발행
- `W08_dock_vrx_run.m` 이 하는 일은 `W08_vrx_run.m` 과 같은 순서임 (D절 표)
  - 도메인 맞춤 → 토픽 확인 → RTF 측정과 **스폰 위치 실측** → 페이싱 설정 → 실행 → 결과 그림 → 오프라인 대조
  - 1단계에서 **선수추진기 토픽** `/wamv/thrusters/bow/thrust` 도 함께 확인함. 없으면 거기서 멈추고 띄우는 명령을 알려 줌
- 기준 환경 실측값 (2026-11-01, 측정 RTF 1.040, 420 s 주행 / 벽시계 448 s, 스폰 $(x_{\mathrm{ENU}}, y_{\mathrm{ENU}}) = (-532.0,\ 200.0)$)

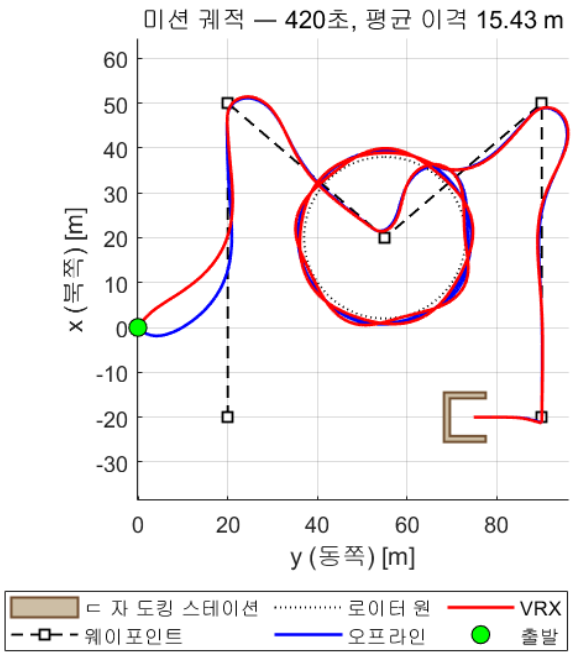
```
===== VRX 연동 — WP -> 로이터 -> WP -> 접안 =====
  로이터 진입              66.6 s
  로이터 종료             227.6 s   (2.00 바퀴)
  접안 시작               288.9 s
  접안 완료               316.7 s   (접안에 27.8 s)
  WP 구간 평균 |y_e|      3.060 m
  로이터 평균 반경오차    1.218 m
  선석 위치 오차          1.476 m   (완료 문턱 1.50 m 을 넘은 순간)
  멈춘 뒤 위치 오차       0.057 m   (완료 +10 s, 그 사이 1.53 m 미끄러짐)
  옆걸음 선수각 오차      0.889 deg  (최대, phase 2 전 구간)
  옆걸음 |u| / |v|        0.177 / 0.600 m/s  (최대, phase 2 전 구간)
  선체 네 점              멈춘 뒤 모두 ㄷ 안
```

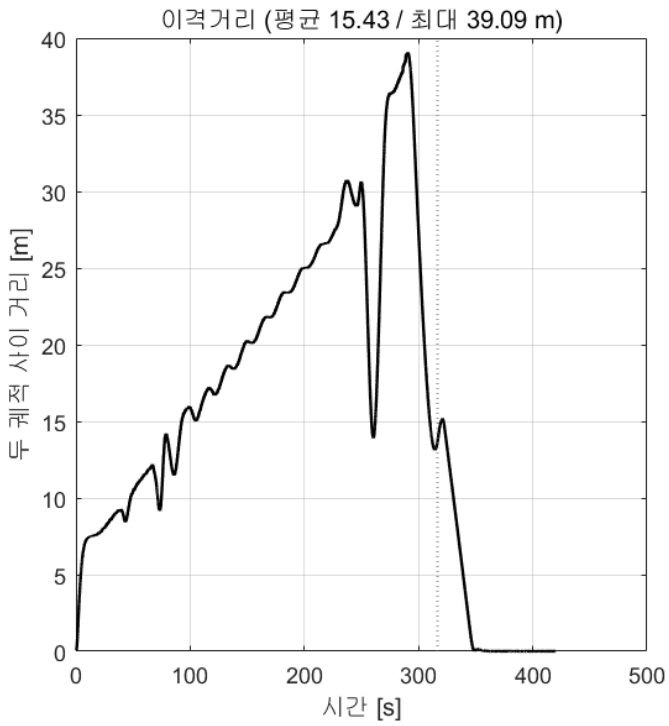



- 세 가지를 수치로 확인함

| 질문 | 답 | 근거 |
|---|---|---|
| 실제로 접안이 되는가 | 됨 | 접안 완료 316.7 s. 미완료가 아님 |
| 선수각이 유지되는가 | 유지됨 | 옆걸음 전 구간 최대 자세오차 **0.889°** |
| 멈춘 자리가 ㄷ 안인가 | 안에 있음 | 선체 네 꼭짓점 모두 베이 다각형 안. 선석까지 **0.057 m** |

### 오프라인과 비교




| 항목 | 오프라인 | VRX | 차이 |
|---|---|---|---|
| 로이터 진입 [s] | 74.6 | 66.6 | −8.0 |
| 로이터 종료 [s] | 247.8 | 227.6 | −20.2 |
| 접안 시작 [s] | 314.6 | 288.9 | −25.7 |
| 접안 완료 [s] | 344.9 | 316.7 | −28.2 |
| 멈춘 뒤 위치 오차 [m] | 0.079 | 0.057 | −0.022 |
| 옆걸음 선수각 오차 [deg] | 0.255 | 0.889 | +0.634 |
| 궤적 평균 이격 [m] | | | 15.43 |
| 궤적 최대 이격 [m] | | | 39.09 |

- 상태 순서 · 회전수 · 접안 성공 여부는 같음. 미션 모델은 이 셋으로 판정함
- **평균 이격 15.43 m 는 경로 오차가 아니라 위상 오차임**
  - 궤적 그림에서 두 선은 겹침. VRX 가 같은 길을 약 28 s 먼저 지남
  - 오른쪽 칸을 보면 이격이 **접안 뒤 0 으로 내려앉음** — 둘 다 같은 선석에 섰다는 뜻임
  - → 궤적이-겹치면-이격은-오차가-아니라-지연이다
- 선수각 오차가 VRX 에서 0.6° 큰 이유 — odometry 가 약 10 Hz 로 들어오는데 제어는 20 Hz 임 (D-3 의 note 와 같은 원인). 1° 안이면 순수 옆걸음으로 봄

**✕ VRX 를 끝낼 때 반드시 정리한다**

```
source ~/vrx_ws/install/setup.bash && ros2 run vrx_llm_dock cleanup
```

- 남은 `gz sim` 프로세스가 수십 GB 까지 자라 메모리가 모자라게 됨

## 확인할 것

- [ ] `W08_dock_setup` 한 줄로 접안까지 끝났는가 — `접안 완료` 줄에 시각이 찍혔는가

- [ ] 로이터 진입·종료 시각이 실습 A 와 같은가 (앞의 세 상태를 건드리지 않았다는 확인)
- [ ] 실시간 화면 ① 의 계단이 **네 번** 바뀌는가
- [ ] 옆걸음 구간에서 선수각 오차가 1° 안인가
- [ ] 확대 그림에서 선체가 ⊏ 안에 들어가 섰는가
- [ ] `bow_on = 0` 으로 두면 접안이 실패하는가

---

# 마무리

## 이번 주차 요약

| 순서 | 한 일 | 확인 |
|---|---|---|
| 1 | Stateflow 차트 작성 | 상태 3개, 전이 4개 |
| 2 | 두 유도법칙 전환 | `ModeSwitch` |
| 3 | 인덱스 잠금 | 로이터 중 인덱스가 평평 |
| 4 | 대수 루프 끊기 | `ModeDly` |
| 5 | 전 구간 실행 | 1→2→1→3 |
| 6 | VRX 연동 | 오프라인과 대조 |
| 7 | 미션 끝에 접안 붙이기 (실습 E) | 상태 4개, 전이 5개 · `BerthRef` |
| 8 | 순수 옆걸음 접안 | 1→2→1→3→4, 멈춘 뒤 0.079 m |
| 9 | Stateflow 차트의 실행 순서 (1-9절) | 한 스텝 네 단계 · 전이 시각 74.6 / 247.9 / 314.6 / 344.9 s |

---

## 수업 진도 체크

### 이론 이해

- ☐ 상태·전이·전이조건·진입동작을 각각 설명할 수 있음
- ☐ `if` 문 대신 상태기계를 쓰는 이유를 말할 수 있음
- ☐ **상태기계**(개념)와 **Stateflow**(도구)의 차이를 말할 수 있음
- ☐ 이번 주차에 작성한 세 상태와 네 전이를 그림으로 그릴 수 있음
- ☐ **로이터 중 인덱스를 잠그지 않으면** 무슨 일이 생기는지 앎
- ☐ 로이터 중심을 **상수로 고정**하는 이유를 앎
- ☐ 회전수를 모드에 묶어 초기화하는 이유를 앎
- ☐ **대수 루프**가 왜 생기고 어떻게 끊는지 설명할 수 있음
- ☐ 로이터를 마친 배가 경로 밖에 있고, LOS 가 다시 끌어온다는 것을 앎
- ☐ **유도법칙으로는 배를 세울 수 없고** 위치제어가 필요한 이유를 말할 수 있음
- ☐ 문 $g_p$ · $g_u$ 하나로 두 모드가 갈린다는 것을 앎 (1-8절)
- ☐ 접안 3단계(접근점 → 자세 맞춤 → 옆걸음)의 전이 조건을 각각 말할 수 있음
- ☐ **순수 옆걸음**이 성립하려면 선수추진기가 필요한 이유를 앎
- ☐ 차트가 한 스텝에 하는 네 가지(입력 읽기 → 나가는 전이 검사 → 전이 하나 실행 → `mode` 출력)를 순서대로 말할 수 있음
- ☐ 네 번의 전이 조건 `at_loiter` · `turns` · `at_end` · `phase` 가 각각 **어느 블록에서** 만들어지는지 말할 수 있음
- ☐ `WpFollow` 에서 전이 둘이 동시에 참일 때 **검사 순서**가 결과를 가른다는 것을 앎
- ☐ 조건이 참이 된 뒤 **한 스텝(0.05 s) 늦게** 상태가 바뀌는 이유를 말할 수 있음

### 실습 완료

- ☐ `W08_setup` 실행만으로 미션이 끝까지 돌았음
- ☐ 실시간 창 **네 칸의 뜻**을 알고, 왼쪽 칸에서 **배 색이 세 번 바뀌는 것**을 봤음

- [ ] 실시간 창 ① 의 **상태 계단이 세 번 바뀌고** 제목에 전이 시각 세 개가 찍히는 것을 봤음
- [ ] 실시간 창 ① 제목의 **바퀴 수가 로이터 구간에서 2.00 까지 오르는 것**을 봤음
- [ ] 로이터에서 빠져나온 뒤 왼쪽 칸의 초록 원이 **웨이포인트를 다시 집는 것**을 봤음
- [ ] 미션 상태 그래프가 1→2→1→3 인 것을 확인했음
- [ ] 웨이포인트 인덱스가 로이터 구간에서 평평한 것을 확인했음
- [ ] Stateflow 차트를 열어 상태와 전이조건을 읽었음
- [ ] 사본 `W08_my` 에 차트를 직접 그려 74.7 s · 248.0 s · 314.4 s 를 다시 얻었음
- [ ] `loiter_wp` , `loiter_turns` , `p_c` 를 바꿔 봤음
- [ ] VRX 에서 같은 미션을 실행했음
- [ ] `W08_dock_setup` 한 줄로 **접안까지** 끝났음 (실습 E)
- [ ] 실습 E 의 로이터 진입·종료 시각이 실습 A 와 같은 것을 확인했음
- [ ] 실시간 창 ① 의 상태 계단이 **네 번** 바뀌는 것을 봤음
- [ ] 확대 그림에서 선체가 ⊏ 안에 들어가 선 것을 봤음
- [ ] `bow_on = 0` 으로 두면 접안이 실패하는 것을 확인했음

### 관찰 기록

- [ ] 상태 전이 시각을 기록했음
- [ ] 구간별 오차를 나눠 기록했음
- [ ] 로이터 직후 경로 복귀 과정을 그림으로 남겼음
- [ ] 접안 **멈춘 뒤 위치 오차**와 **옆걸음 선수각 오차**를 수치로 기록했음
- [ ] 멈춘 자리와 ⊏ 벽 사이 여유를 네 방향으로 기록했음

---

## 과제 7 — 미션 상태기계

- **제출 기한**: 9주차 수업 전

### ① 상태 전이 기록

| 사건 | 시각 [s] | 그때 `mode` | 그때 `idx` |
|---|---|---|---|
| 시작 | 0 | | |
| 로이터 진입 | | | |
| 로이터 종료 | | | |
| 임무 종료 | | | |

- 미션 상태 그래프와 웨이포인트 인덱스 그래프를 함께 제출

### ② 인덱스 잠금을 풀어 보기

- 기본 경로에서는 잠금을 풀어도 결과가 같음 (1-4절). 먼저 WP4 를 로이터 원 위로 옮김
  - `W08_setup.m` 에서 `wp_north(4) = 4; wp_east(4) = 47;` 로 바꾼 뒤 실행 → 1-4절 표의 "잠금 있음" 열과 비교
- 사본 모델(B-2 의 `W08_my` )의 `Guidance/WpManager` 에서 인덱스 전진 조건의 `(mode < 1.5) &&` 를 **지우고** 실행
- 제출
  - 두 경우의 궤적 그림과 웨이포인트 진행 그래프
  - `idx` 가 로이터 중 몇 초에 바뀌었는지, WP4 를 들렀는지
  - 잠금이 없는데도 임무가 **더 빨리** 끝나는 이유 (3줄)

- **끝나면 조건과 웨이포인트를 원래대로 되돌려 놓을 것**

### ③ 로이터 파라미터 스윕

| `loiter_radius` | `loiter_turns` | 로이터 소요시간 [s] | 평균 반경오차 [m] | 복귀 후 최대 $\lvert y_e \rvert$ [m] |
|---|---|---|---|---|
| 10 | 2 | | | |
| 18 | 2 | | | |
| 25 | 2 | | | |

- 복귀 후 최대 $\lvert y_e \rvert$ 는 `W08_plot` 이 내지 않음. `W08_setup` 을 돌린 뒤 아래로 구함
  - 구간: 로이터 마지막 샘플 다음부터 임무 종료까지 ( `mode` = 1)

```
t   = out.log_y_e.Time;   ye = out.log_y_e.Data;   md = out.log_mode.Data;
k1  = find(md > 1.5 & md < 2.5, 1, 'last') + 1;   % 로이터 직후
k2  = find(md > 2.5, 1);                           % 임무 종료
if isempty(k2), k2 = numel(t); end
ye_max = max(abs(ye(k1:k2)))
```

- 기본값(반경 18 m)에서는 1-6절의 7.60 m 가 나와야 함
- **반경이 커지면 복귀가 더 어려워지는가?** 수치로 답하시오

### ④ 로이터를 두 번 하기

- 상태기계를 고쳐 **두 지점에서 각각 로이터링**하게 만드시오
- 고칠 곳: 차트 `W08_my/Mission/MissionFSM` , 도착 판정 `W08_my/Guidance/WpManager`
  - 원본 대신 B-2 절에서 만든 사본 `W08_my` 에서 작업할 것 — `build_w08_models` 는 원본을 지우고 다시 만듦
  - 차트 편집 절차는 B-2 절과 부록 A1 M-5 절
- 방법은 자유 (상태 추가 / 배열화 / 카운터 사용)
- 상태 전이도를 그리고 실행 결과를 제출

### ⑤ 오프라인 ↔ VRX

- 같은 설정으로 두 모델을 실행
- 상태 전이 시각과 구간별 오차를 표로 비교

### ⑥ 분석 (8~12줄)

1. 상태기계로 나눴을 때 얻은 것과 잃은 것은 무엇인가
2. 대수 루프를 단위지연으로 끊었음. 이 지연이 문제가 되는 경우는 언제인가
3. 로이터 직후 경로 복귀가 임무 전체 성능에 미치는 영향은
4. Term Project 의 미션(이안→회피→위치유지→도킹)을 상태기계로 그려 보시오

### 평가 기준

| 항목 | 배점 |
|---|---|
| ① 전이 기록 | 15% |
| **② 인덱스 잠금 실험** | 20% |
| ③ 파라미터 스윕 | 20% |
| **④ 로이터 두 번 — 상태기계 직접 수정** | 25% |
| ⑤·⑥ 대조와 분석 | 20% |

---

## 막혔을 때

| 증상 | 원인 | 해결 |
|---|---|---|
| `먼저 W08_setup 을 실행하십시오` | 워크스페이스에 변수 없음 | `W08_setup` 먼저 |
| **대수 루프 오류** | `mode` 되먹임에 지연이 없음 | `ModeDly` 확인 |
| 로이터를 안 하고 지나감 | `at_loiter` 가 안 뜸 | `loiter_wp` 값과 `R_LOS` 확인 |
| 로이터에서 안 나옴 | `turns` 가 안 오름 | 원에 도달했는지 반경 그래프 확인 |
| 로이터 뒤 웨이포인트 하나를 건너뜀 | 인덱스 잠금이 풀림 (원이 다음 웨이포인트를 지나감) | `WpManager` 의 `(mode < 1.5)` 확인 |
| 로이터 원이 이동함 | 중심을 인덱스로 잡음 | `loiter_xc/yc` 는 상수여야 함 |
| 복귀 후 경로를 크게 벗어남 | 로이터 반경이 큼 | `Delta` 를 키우거나 반경을 줄일 것 |
| VRX 에서 배가 멈춤 | 육지에 얹힘 | 로이터 원이 안전 수역 안인지 |
| 주행거리와 `u` 가 2배 차이 | 페이싱 ≠ RTF | RTF 재측정 |
| 접안으로 안 넘어감 (실습 E) | `at_end` 가 안 뜸 | 마지막 웨이포인트 수락 반경 `R_LOS` 확인 |
| 접안 중 배가 제자리에서 돎 | 선수추진기 토픽이 없음 — 기본 WAM-V 로 띄움 | `run_vrx_bow.sh` 로 다시 띄울 것 (E-4) |
| 옆걸음이 아니라 비스듬히 들어감 | 자세가 맞기 전에 2단계로 넘어감 | `BerthRef` 의 $5° \cdot \lvert u \rvert < 0.20$ 조건 확인 |
| 멈춘 뒤에도 벽을 밀고 들어감 | 스웨이 PI 적분기가 **얼어붙음** — 문을 닫아 오차를 0 으로 만든 경우 | `gate_d` 는 `Dock` 이후 **열린 채로 둘 것** (1-8절 warning) |
| 접안 완료가 안 찍힘 | 선석까지 거리가 `R_stop` 안에 못 들어옴 | `L_app` 를 줄이거나 `v_max` 를 확인 |
| 실습 E 에서 배가 육지에 얹힘 | `dock_deg` 나 `berth_side` 를 바꿔 ㄷ 이 안전 수역을 벗어남 | 1-8절 여유 표의 범위 안으로 되돌릴 것 |

---

## 참고 자료

### 이 주차의 원본

- `UGV_WpFollowingAndLoitering/MILS/UGV_Control_StateFlow_260529.slx` — 4상태 미션 FSM, `KAIST_CircularVF`, `calculate_turns`
- `init_UGV_collision_avoidance_scripts_stateflow_260529.m` — 결과 플로팅과 회전수 집계

### 공식 문서

- Stateflow 시작하기 — https://www.mathworks.com/help/stateflow/getting-started-with-stateflow.html
- 대수 루프 — https://www.mathworks.com/help/simulink/ug/algebraic-loops.html

### 볼트 내 문서

- W05_웨이포인트_유도_atan2와_LOS — LOS 유도, 내부루프, **실시간 화면 네 칸의 원형** (A절)
- W07_한점을_중심으로_도는_로이터링 — 벡터필드, 회전수
- 오프라인-모델이-시뮬레이터와-같아야-게인이-옮겨간다
- 궤적이-겹치면-이격은-오차가-아니라-지연이다 — D-2 · D-3 의 전이 시각 차이를 읽는 법

### 배포 파일

| 파일 | 내용 |
|---|---|
| `W08_simulink/W08_setup.m` | **학생이 고치는 설정 파일. 실행하면 바로 돎** |
| `W08_simulink/W08_0_offline.slx` | 오프라인 미션 |
| `W08_simulink/W08_1_vrx.slx` | VRX 연동 |
| `W08_simulink/W08_plot.m` | 상태별 성능 그림 |
| `W08_simulink/W08_animate.m` | 실시간 화면 네 칸 그리기 — 항적(상태별 색) · 상태 타임라인 · 지령 대비 응답 · 추력. 오프라인·VRX 공용 (모델 안 `Animate` 상자가 호출) |
| `W08_simulink/W08_anim_capture.m` | 그 화면을 조건과 함께 PNG 로 재현 ( `img/W08_anim_offline.png` · `W08_anim_vrx.png` ) |
| `W08_simulink/W08_vrx_run.m` | VRX 연동 실행 — 토픽 확인 · RTF 측정 · 페이싱 · 결과 그림 · 오프라인 대조 |
| `W08_simulink/build_w08_models.m` | 모델 재생성 (Stateflow 차트 포함) |

실습 E(접안) 전용 — 위의 파일과 겹치지 않음

| 파일 | 내용 |
|---|---|
| `W08_simulink/W08_dock_setup.m` | **실습 E 의 설정 파일. 실행하면 바로 돎** |
| `W08_simulink/W08_2_dock.slx` | 오프라인 미션 + 접안 |
| `W08_simulink/W08_3_dock_vrx.slx` | VRX 연동 미션 + 접안 |
| `W08_simulink/W08_dock_plot.m` | 상태별 성능 그림 + ㄷ 자 확대 + 접안 지표 표 |
| `W08_simulink/W08_dock_animate.m` | 실시간 화면 네 칸 (상태 색 4개 · ㄷ 자 · 접안 단계) |
| `W08_simulink/W08_dock_init.m` | 모델 `InitFcn` — 시계가 돌기 **전에** 실시간 창을 지음 |
| `W08_simulink/W08_dock_done.m` | 모델 `StopFcn` — 직접 Run 했을 때만 결과 그림을 냄 |
| `W08_simulink/W08_dock_set_callbacks.m` | 위 두 콜백을 모델에 심음 |
| `W08_simulink/W08_dock_patch.m` | ㄷ 자 도킹 스테이션 다각형 (6주차 사본 — 8주차를 자립시킴) |
| `W08_simulink/W08_dock_vrx_run.m` | VRX 실행 — 선수추진기 토픽 확인 · RTF · 스폰 실측 · 대조 |
| `W08_simulink/build_w08_dock.m` | 실습 E 모델 재생성 ( `>> build_w08_dock` ) |

---

## 다음 주 예고

- **9주차** — 라이다 장애물회피. 제어 트랙의 마지막 주차이고, **제어기는 5주차 것을 한 줄도 안 바꿈**
  - 바뀌는 자리는 유도단 하나뿐임 — LOS 와 파이썬 회피기 **둘** 중 하나를 고르는 상자가 생김
  - 회피 명령은 ROS 2 토픽 `/vfh/command` 로 들어오고, 쓸지 말지는 `/vfh/oa_flag` 가 정함
- **9주차는 중간 발표 주간임** — 팀별 Term Project 제안서 초안을 준비할 것
- **10주차부터** — 인지 트랙 (10주차 LiDAR 신호처리, 11주차 영상처리와 LiDAR-카메라 융합)
- 이번 주차에 작성한 상태기계에 **회피 상태**를 끼워 넣는 일
  - 9주차: 회피 노드를 LOS 유도단과 결합 — 이번 주차 FSM 의 `WP` 상태 안에서 갈라짐
  - 12주차: 팀 미션 매니저에 회피 상태를 삽입
- Term Project 의 전체 미션도 같은 방식으로 조립함

> **★ 중간 발표**
>
> 평가의 **15%** 임. 전반부 데모(8주차까지 만든 것)와 Term Project 제안서를 냄
> 자세한 내용은 평가와-팀운영 과 Term-Project-명세 를 볼 것